BASES DE DONNES

- Généralités
- Les bases de données relationnelles
- Normalisation des relations
- Le Modèle entité association
- L'algèbre relationnel
- Le langage SQL

Par GBO Hervé ***Tél*** : 95564920 /97188083 ***e-mail*** : hervegbo@yahoo.fr

CHAPITRE I : GENERALITES

I - INTRODUCTION

Dans les formes traditionnelles de l'information, les données stockées sur des supports constituent des fichiers. Ces fichiers, construits pour répondre aux besoins des applications informatiques possèdent la plupart du temps des éléments communs, des relations qui ne sont pas exploitées du fait qu'ils sont utilisés isolement et indépendamment les uns des autres. Cette manière d'organiser les informations présente trois inconvénients :

- Une grande complexité à cause du grand nombre de fichiers et à leur création qui dépend des besoins particuliers de chaque application.
- La redondance des informations à cause de la duplication sur plusieurs fichiers de certaines données. Ce qui est d'une part coûteux en moyen de stockage et d'autre part pose le problème de cohérence car les mises à jour d'une même donnée localisée sur plusieurs fichiers utilisés par des applications différentes, ne peut pas toujours être simultanée.

Par ailleurs, la prise de décision est une part importante de la vie d'une société. Mais elle nécessite d'être bien informée sur la situation et donc d'avoir des informations à jour et disponibles immédiatement.

D'autre part, les utilisateurs, ne veulent plus de systèmes d'information constitués d'un ensemble de programmes inflexibles et de données inaccessibles à tout non spécialiste; ils souhaitent des systèmes d'informations globaux, cohérents, directement accessibles (sans qu'ils aient besoin soit d'écrire des programmes soit de demander à un programmeur de les écrire pour eux) et des réponses immédiates aux questions qu'ils posent. Il a donc fallu rechercher des solutions tenant compte à la fois des désirs des utilisateurs et des progrès techniques. Cette recherche a abouti au concept de base de données.

II - DEFINITIONS ET AVANTAGES D'UNE BASE DE DONNEE

Définition

Une base de données est un ensemble d'informations qui modélise les objets d'une partie du monde réel, qui peut servir de support à une application informatique et qui est exhaustif, non redondant, structuré et persistant.

- ***La partie du monde réel*** est le sujet étudié ou le domaine circonscrit par la base de données. exemple : Gestion de la scolarité des étudiants, Gestion des patients d'un hôpital, etc.
- On appelle ***objets d'une partie du monde réel***, toute entité, ayant un sens pour le sujet étudié, qui peut lui fournir des informations et qui possède au moins deux occurrences (ou exemplaires). On distingue des objets abstraits et des objets réels. Exemple, dans le cas de la gestion de la scolarité des étudiants on distingue les objets réels suivants : Etudiants, Profs, Salle de classe, etc. et les objets abstraits suivants : Matière, Filière, etc.
- ***Modéliser (dessiner, structurer, schématiser) les objets d'une partie du monde réel*** revient à organiser les données suivants les différentes entités et à définir les liens entre ces entités.

Avantages

Les avantages que l'on peut attendre d'une base de données sont nombreux :

- *Saisie unique* : Les données sont saisies une seule fois et stockées indépendamment des programmes qui doivent les traiter.
- *Mise à jour unique* : Comme une donnée n'est stockée qu'une seule fois, le problème souvent mal résolu de la mise à jour de cette donnée dans différents fichiers ne se pose plus. Les différents traitements qui utiliseront cette donnée seront toujours très cohérents.

- *Gain de place au stockage* : La suppression des redondances réduit le volume nécessaire au stockage.
- *Accès plus facile à l'information* : Chaque donnée est stockée indépendamment des programmes qui l'utilisent. Elle est disponible pour les utilisateurs potentiels qui interrogeront la base. L'ensemble des informations de l'Entreprise peut ainsi être partagé entre les services sans qu'aucun n'en conserve l'exclusivité.
- *Possibilité d'évolution* : Une base de donnée bien conçue est indépendante des traitements. Elle peut en principe permettre la prise en compte facile de nouvelles applications si les possibilités d'accès aux données ont été bien prévues lors de sa conception.

III - LES SGBD

Un système de gestion de base de données est un logiciel qui permet de décrire, modifier, interroger et administrer les données d'une base de données.

1. Objectifs des systèmes de gestion de bases de données

Les bases de données et les systèmes de gestion de bases de données ont été créés pour répondre à un certain nombre de besoins et pour résoudre un certain nombre de problèmes.
Les objectifs d'un SGBD sont les suivants :

- ***Indépendance physique***
  La façon dont les données sont définies doit être indépendante des structures de stockages utilisées.
- ***Indépendance logique***
  Un même ensemble de données peut être vu différemment par des utilisateurs différents. Toutes ces visions personnelles des données doivent être intégrées dans une vision globale.
- ***Manipulations des données par des non informaticiens***
  Il faut pouvoir accéder aux données sans savoir programmer ce qui signifie des langages "quasi naturels" comme SQL.
- ***Efficacité des accès aux données***
  Ces langages doivent permettre d'obtenir des réponses aux interrogations en un temps "raisonnable". Ils doivent donc être optimisés et, entre autres, il faut un mécanisme permettant de minimiser le nombre d'accès disques. Tout ceci, bien sûr, de façon complètement transparente pour l'utilisateur.
- ***Administration centralisée des données***
  Des visions différentes des données (entre autres) se résolvent plus facilement si les données sont administrées de façon centralisée.
- ***Non redondance des données***
  Afin d'éviter les problèmes lors des mises à jour, chaque donnée ne doit être présente qu'une seule fois dans la base.
- ***Cohérence des données***
  Les données sont soumises à un certain nombre de contraintes d'intégrité qui définissent un état cohérent de la base. Elles doivent pouvoir être exprimées simplement et vérifiées automatiquement à chaque insertion, modification ou suppression des données.
- ***Accès concurrent aux données***
  Il s'agit de permettre à plusieurs utilisateurs d'accéder aux mêmes données au même moment. Si ce problème est simple à résoudre quand il s'agit uniquement d'interrogations et quand on est dans un contexte mono utilisateur, cela n'est plus le cas quand il s'agit de modifications dans un contexte multi utilisateurs. Il s'agit alors de pouvoir :

    - permettre à deux (ou plus) utilisateurs de modifier la même donnée "en même temps" ;

- assurer un résultat d'interrogation cohérent pour un utilisateur consultant une table pendant qu'un autre la modifie.

- ***Sécurité des données***
  Les données doivent pouvoir être protégées contre les accès non autorisés. Pour cela, il faut pouvoir associer à chaque utilisateur des droits d'accès aux données.
- ***Résistance aux pannes***
  Que se passe-t-il si une panne survient au milieu d'une modification, si certains fichiers contenant les données deviennent illisibles? Les pannes, bien qu'étant assez rares, se produisent quand même de temps en temps. Il faut pouvoir, lorsque l'une d'elles arrive, récupérer une base dans un état "sain". Ainsi, après une panne intervenant au milieu d'une modification deux solutions sont possibles : soit récupérer les données dans l'état dans lequel elles étaient avant la modification, soit terminer l'opération interrompue.

Il existe des SGBD de complexité variable qui possèdent tout ou partie des propriétés ci-dessus. Prenons en exemple deux produits assez caractéristiques : le SGBD relationnel Oracle et le SGBD relationnel Access. Le SGBD Oracle est un SGBD relationnel utilisé pour des applications critiques et qui offre un maximum des caractéristiques présentées ici. Le SGBD Access est un SGBD dans le monde de l'informatique individuelle qui présente l'avantage d'une grande facilité d'utilisation et qui peut convenir à des applications de taille réduite ou moyenne. L'aspect convivial de ce dernier étant évident. En revanche, les niveaux de performance et de sécurité ne sont pas comparables.

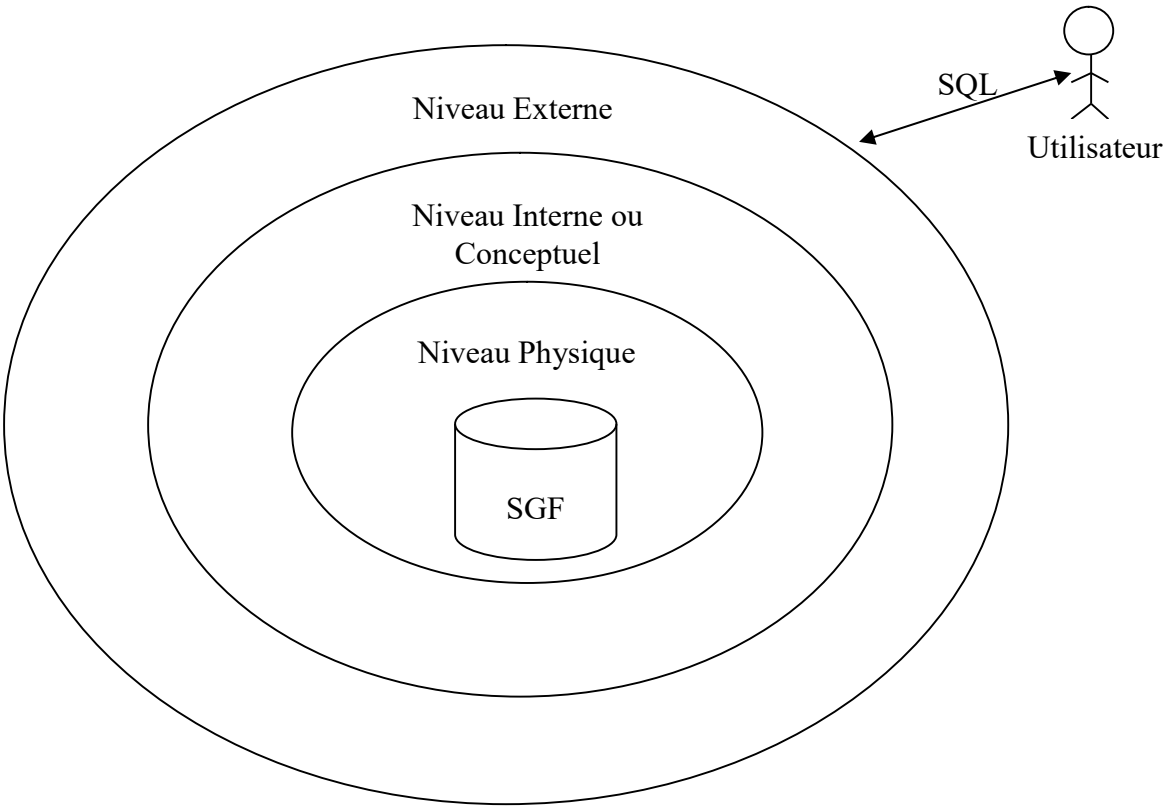
2. Architecture des SGBD

Les SGBD reposent sur trois niveaux d'abstraction qui assurent l'indépendance logique et physique des données, autorisent la manipulation de données, garantissent l'intégrité des données et optimisent l'accès aux données. L'architecture d'un SGBD a trois niveaux :

- ***Le niveau externe.***
  Il regroupe toutes les possibilités d'accès aux données par les différents usagers. Ces accès, éventuellement distants, peuvent se faire via différents types d'interfaces et langages plus ou moins élaborés. Ce niveau détermine le schéma externe qui contient les vues des utilisateurs sur la base de données c'est à dire le sous-ensemble de données accessibles ainsi que certains assemblages d'information et éventuellement des informations calculées. Il peut donc exister plusieurs schémas externes représentant différentes vues sur la base de données avec des possibilités de recouvrement.

- ***Le niveau conceptuel.***
  Il correspond à la vision des données générale indépendante des applications individuelles et de la façon dont les données sont stockées. Cette représentation est en adéquation avec le modèle de données utilisé. Dans le cas des SGBD relationnels, il s'agit d'une vision tabulaire où la sémantique de l'information est exprimée en utilisant les concepts de relation, attributs et de contraintes d'intégrité. Le niveau conceptuel est défini au travers du schéma conceptuel.

- ***Le niveau physique.***
  Il regroupe les services de gestion de la mémoire secondaire. Il s'appuie sur un système de gestion de fichiers pour définir la politique de stockage ainsi que le placement des données. Cette politique est définie en fonction des volumes de données traitées, des relations sémantiques entre les données ainsi qu'en fonction de l'environnement matériel disponible.

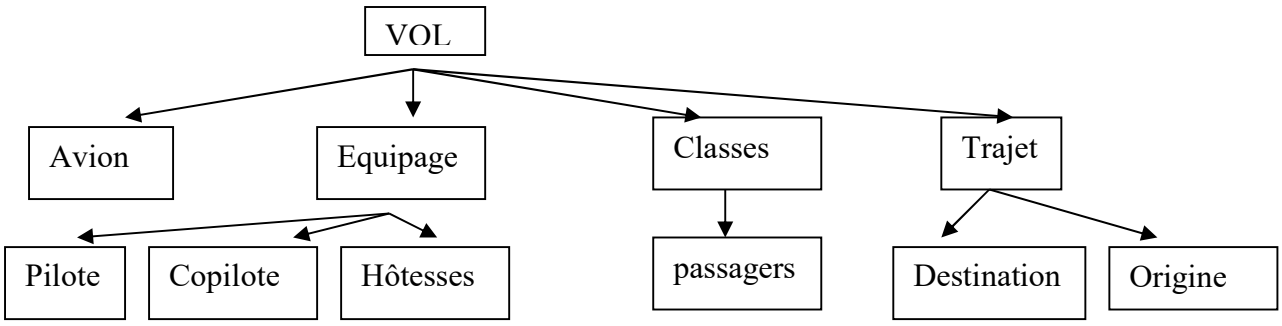


IV- LES DIFFERENTS MODELES D'UNE BASE DE DONNEES

Le principal objectif des bases de données est de rendre indépendant les données vis à vis des applications. L'accès à ces données par des applications pose des problèmes. Ainsi, pour simplifier ces problèmes d'accès, plusieurs modèles logiques de base de données et de systèmes de gestion de base de donnée ont vu le jour. On distingue par ordre chronologique :

1 - Le modèle hiérarchique

Historiquement le premier, il consiste à organiser les données de façon arborescente; ce qui constitue, une structure simple à gérer. Cette structure est une hiérarchie ou chaque élément n'a qu'un supérieur. Le nombre de connexion est limité: il n'y en a pas entre les branches de même niveau.



Ce modèle ne permet que des interrogations simples. Par exemple, quel est le trajet du vol 512 ou quel est le pilote du vol 304. Mais, il n'est pas aisé de savoir sur quel vol est inscrit le passager Mahigôh.

2 - Le Modèle Réseau

Le modèle réseau est une extension du modèle précédent: il permet d'établir des connexions entre les différents éléments. De cette manière, on dispose d'un plus grand nombre d'interrogations possibles mais, elles doivent être toujours prévues lors de la construction de la base de données.

3- Le Modèle Relationnel

Le modèle relationnel Permet de se libérer de la contrainte suivante : Connaître à l'avance les interrogations que l'on effectuera. Ainsi, les données sont stockées sous la forme de relation dans des tables. Ce type de structure permet d'établir des connexions au moment de l'exécution. On pourra donc effectuer toute sorte d'interrogations plus ou moins complexes.
L'accès aux bases de données relationnelles s'effectue en appliquant les trois opérations de base suivantes: la projection, la sélection et la jointure .
Avec le modèle relationnel, il est nécessaire de convertir les données d'application sous forme de tables. L'interface entre une application et une base de données est effectuée à l'aide de langages spécifiques, dont le plus connu est SQL

Chapitre II – Les bases de données relationnelles

Le modèle relationnel a été proposé par E.F. Codd en 1970. Il est souvent considéré comme le plus simple et le plus élégant des modèles. Sa simplicité est due à une vision tabulaire des données très intuitive. En effet, dans ce modèle, les données sont stockées dans des tables, sans préjuger de la façon dont les informations sont stockées dans la machine. Un ensemble de données sera donc modélisé par un ensemble de tables ou relations.
Le succès du modèle relationnel auprès des chercheurs, concepteurs et utilisateurs est dû à la puissance et à la simplicité de ses concepts. En outre, contrairement à certains autres modèles, il repose sur des bases théoriques solides, notamment la théorie des ensembles.

Les objectifs du modèle relationnel :

- proposer des schémas de données faciles à utiliser,
- améliorer l'indépendance logique et physique,
- mettre à la disposition des utilisateurs des langages de haut niveau pouvant éventuellement être utilisés par des non informaticiens (SQL),
- optimiser les accès à la base de données,
- améliorer l'intégrité et la confidentialité,
- fournir une approche méthodologique dans la construction des schémas,
- Permettre un haut degré d'indépendance entre les applications (programmes, interfaces) et la représentation interne des données (fichiers, chemins d'accès)
- Etablir une base solide pour traiter les problèmes de cohérence et de redondance des données

De façon informelle, on peut définir une base de données relationnelle comme étant une base dans laquelle

- Les données sont organisées sous forme de tableaux à deux dimensions, encore appelées relations et chaque ligne un uplet, une occurrence, un enregistrement ou un tuple,
- les données sont manipulées par des opérateurs de l'algèbre relationnelle,
- l'état cohérent de la base est défini par un ensemble de contraintes d'intégrité.

Le modèle relationnel présente également de nombreux avantages dus au fait qu'il soit basé sur la théorie des ensembles : Langage de manipulation des données ensemblistes grâce à l'algèbre relationnelle et grâce à des langages assertionnels qui permettent de spécifier ce que l'on souhaite obtenir sans dire comment l'obtenir. Le SGBD est responsable de la politique d'exécution des requêtes.

Au modèle relationnel est associée la théorie de la normalisation des relations qui permet de se débarrasser des incohérences au moment de la conception d'une base de données.

I. *Concepts de base*

Attribut **:** C'est la plus petite information (atomique) qui a un sens pour le domaine d'étude.

Relation **:** c'est un ensemble d'attributs qui caractérisent un individu (Etudiant), un objet (Filière) ou un évènement (Composer). Elle peut être considérée comme un tableau dans lequel chaque colonne correspond à un attribut et porte un nom et chaque ligne représente un enregistrement, tuple, uplet ou une occurrence.

Schéma de relation : Un schéma de relation R, est utilisé pour décrire une relation et est noté :
R(A1, A2, …, An) ou R est le nom de la relation et Ai les attributs qui décrivent la relation.

Illustration ou extension d'une relation : c'est la représentation d'une relation sous forme de tableau avec comme entête de colonne, les attributs et comme ligne, les occurrences de la relation. Exemple si on considère la relation Etudiant de schéma :

Etudiant(Matricule, nom, prénom, sexe, codfil)

Une extension de cette relation serait le tableau suivant :

Matricule	Nom	Prénom	Sexe	Codfil
1	Magengo	Guttembert	M	IG
2	Jolie	Joliette	F	MMI
3	Mahigôh	Bitôh	M	IIR
4	Gros	Séchard	M	IIR

Clé primaire d'une relation **: c'**est un ensemble minimum d'attributs dont la connaissance des valeurs permet d'identifier un enregistrement unique de la relation considérée. Une clé primaire est invariante dans le temps. Lors de la définition d'un schéma cette clé primaire est soulignée.

Clés secondaires, externes ou étrangères d'une relation **:** ce sont des attributs qui sont clé primaire de relation et qui se retrouvent dans d'autres relations permettant ainsi d'établir un lien entre les relations. Lors de la définition d'un schéma ces clés étrangères sont précédées du symbole #.

Base de données relationnelles **:** Base de données dont le schéma est un ensemble de schémas de relations normalisés (en troisième forme normale) et dont les occurrences sont les tuples de ces relations.

Système de gestion de bases de données relationnelles **:** C'est un logiciel supportant le modèle relationnel, et qui peut manipuler les données avec des opérateurs relationnels.

Les contraintes d'intégrité : Une contrainte d'intégrité est une propriété, invariante dans le temps qui permet de contrôler l'intégrité des données.
Il existe différents types de contraintes d'intégrité:

- les contraintes liées au modèle (pas de doublons dans une relation.);
- les contraintes de domaine (nb_heure < 100; Note d'un étudiant comprise entre 0 et 20);
- les contraintes référentielles dites de clé étrangère qui imposent que la valeur d'attribut de la relation r_1 apparaissent comme valeur de clé dans une autre relation r_2.

II. - ***Normalisation des relations***

Pour éviter des redondances, pour faciliter les modifications, pour obtenir des structures de données plus claires, il est conseillé de normaliser les relations. La normalisation consistera à décomposer une relation sans perte d'informations, à partir de la notion de dépendance fonctionnelle. L'objectif de la normalisation est d'aboutir à un schéma conceptuel représentant les entités et les associations canoniques du monde réel.

1. Les dépendances fonctionnelles

a. Définition et notation

- On dit qu'un attribut A est en dépendance fonctionnelle avec un autre attribut B (Ou un attribut B dépend fonctionnellement d'un attribut A) et on note : **A→B** si pour toute valeur de A, il ne peut correspondre au plus une valeur de B

- On dit que deux attributs A et B sont en dépendance fonctionnelle avec un autre attribut X et on note : **A,B → X** si pour toute valeur de A, il peut correspondre plusieurs valeurs de B et réciproquement, pour toute valeur de B, il peut correspondre plusieurs valeurs de A et pour toute valeur de A et de B, il correspond au plus, une valeur de X

Les attributs se trouvant à gauche de la flèche sont appelés ***Source*** de la dépendance fonctionnelle et sont généralement des identifiants d'objets.
Les attributs se trouvant à droite de la flèche sont appelés ***Cible*** de la dépendance fonctionnelle et peuvent être un identifiant ou un attribut non identifiant.
Exemple : Numéro_matricule → Nom,
Référence_produit → libellé_produit
Référence_produit, numfact → Pu_produit

b. Propriété des dépendances fonctionnelles

Les propriétés les plus utilisées sont :

- ***La réflexivité*** : On parle de réflexivité lorsque tout attribut ou ensemble d'attribut, détermine lui-même ou une partie de lui-même. Exemple, A→A ou A,A→B ou A,B→B

- ***L'augmentation*** : On parle d'augmentation dans une dépendance fonctionnelle dont la source est composée d'au moins 2 attributs, si et seulement si, une partie de la source est en dépendance fonctionnelle avec un attribut en but c'est-à-dire : Si A,B→C on dit qu'il y a augmentation, si A→C ou B→C
  Remarque : Dans une dépendance fonctionnelle, l'attribut source peut être enrichi d'un ou de plusieurs autres attributs sans altérer la dépendance fonctionnelle. En somme, Si A→B alors, on peut écrire A,C→B. La dépendance restera toujours fonctionnelle mais, elle contiendra une augmentation.

- ***L'addition*** : Toutes les dépendances fonctionnelles de même source peuvent être additionnées pour donner une seule dépendance fonctionnelle c'est-à-dire, Si A→B et A→C alors, on peut écrire par addition que A→B,C

- ***Transitivité*** : On dit qu'une dépendance fonctionnelle est une transitivité si elle peut se déduire de deux autres dépendances fonctionnelles telles que : Si A→B et B→C alors on peut déduire par transitivité que A→C.
  En somme, dans une dépendance fonctionnelle, si un attribut en but dépend d'un autre attribut que la source (Ou encore, si deux attributs en but sont en dépendance fonctionnelle) On dit qu'il y a transitivité. Exemple : si A→B,C et (B→C ou C→B), on peut déduire que la dépendance fonctionnelle A→B,C contient une transitivité.

c. Les Dépendances fonctionnelles élémentaires

Une dépendance fonctionnelle est dite élémentaire si et seulement si elle ne contient pas d'augmentation c'est-à-dire si l'attribut en but dépend de toute la clé primaire et non d'une partie de la clé primaire. En somme la dépendance fonctionnelle A,C→B est élémentaire si et seulement si je ne peux avoir A→B ou C→B.
Exemple : Numéro_matricule, Code_matière →Nom n'est pas élémentaire car l'attribut Numéro_matricule permet à lui tout seul de déterminer le Nom. C'est-à-dire qu'on peut avoir Numéro_matricule→Nom

Remarque
Toute dépendance fonctionnelle dont la source n'est pas composée est automatiquement élémentaire.

d. Les Dépendances fonctionnelles élémentaires directes

Une dépendance fonctionnelle élémentaire est dite Directe si et seulement si elle ne contient pas de transitivité c'est-à-dire si l'attribut en but dépend uniquement de la clé primaire en source et non d'un autre attribut en but. En somme la dépendance fonctionnelle élémentaire A →B,C est directe si et seulement si je ne peux avoir C→B ou B→C.
Exemple : la dépendance fonctionnelle élémentaire Numéro_matricule→ code_filière, libellé_filière n'est pas directe car l'attribut code_filière permet de déterminer le libellé de la filière. En effet, pour un code filière, on ne peut faire correspondre qu'un et un seul libellé de filière.

Résumé

Dépendance fonctionnelle (DF)	Au plus une valeur
Dépendance fonctionnelle élémentaire (DFE)	Pas d'augmentation
Dépendance fonctionnelle élémentaire directe (DFED)	Pas de transitivité

2. Les formes normales

La normalisation est une opération qui consiste à décomposer une relation en plusieurs relations ne contenant ni augmentation ni transitivité sans perte d'information. Elle se fait en trois étapes qui sont : la Première forme normale (1FN), la deuxième forme normale (2FN), et la troisième forme normale (3FN)

a – *La 1ère forme normale*

Une relation est en 1ière forme normale si tous ses constituants (attributs) sont simples, si elle admet une clé primaire (qui est composée du plus petit ensemble d'attribut) en dépendance fonctionnelle avec les autres attributs de la relation
En somme, pour qu'une relation soit en première forme normale, il faut lui trouver une clé primaire. Cette clé primaire doit être l'ensemble minimum d'attribut qui soit en dépendance fonctionnelle avec tous les autres attributs de la relation.
Ex : Soit la relation commander(numéro_bon, référence_produit, date_commande, libellé_produit, pu, qté_commandée, Adresse_livraison, Numéro_client, Nom_client).
Mettez la en première forme normale sachant que l'adresse de livraison se compose de la ville, du quartier et de la rue.

Pour mettre une relation en première forme normal, il faut :

- ***Décomposer tous les attributs qui sont décomposables et dont les différentes parties ont un sens pour le domaine étudié***

Adresse de livraison doit donc disparaitre et les attributs ville, quartier et rue doivent être ajoutés

- ***Rechercher la clé primaire en faisant l'inventaire des attributs qui peuvent être clé primaire(les numéro, code et référence) et en éliminant ceux qui sont inutiles c'est-à-dire ceux qui sont but de dépendance fonctionnelle d'un autre attribut et qui n'interviendront pas comme source dans une dépendance fonctionnelle à source composée.***

numéro_bon, Numéro_client, référence_produit sont les attributs qui ressemblent à des clés primaires

numéro_bon→ Numéro_client car un bon de commande n'appartient qu'à un seul client. Tous les attributs ayant comme source de dépendance fonctionnelle numéro_client, seront donc par transitivité en dépendance fonctionnelle avec numéro_bon. On peut donc sortir numéro_client de la liste des attributs qui feront partie de la clé primaire et garder seulment numéro_bon, référence_produit

- ***La dernière étape consiste à vérifier que la clé primaire trouvée est effectivement en DF avec les autres attributs de la relation***

numéro_bon, référence_produit →Numéro_client, date_commande, libellé_produit, pu, qté_commandée, ville, quartier, rue, Numéro_client, Nom_client

On peut donc conclure que la relation commander(**numéro_bon**, **référence_produit**, date_commande, libellé_produit, pu, qté_commandée, ville, quartier, rue, Numéro_client, Nom_client) est en 1FN

b – *la 2ième forme normale*

Une relation est en 2ième forme normale si elle est en 1ière forme normale et si tous ses constituants sont en dépendance fonctionnelle élémentaire avec la clé primaire. C'est à dire que pour être en 2ième forme normale, une relation doit être en 1ière forme normale et tout attribut autre que la clé primaire ne doit pas dépendre d'une partie de la clé primaire. Plus simplement, on peut dire qu'une relation est en deuxième forme normale si elle est en première forme normale et si elle ne contient pas d'augmentation.
Ex : La relation commander est - elle en 2ième forme normale ? Si non, mettez la en 2ième forme normale.

numéro_bon, référence_produit →Numéro_client, date_commande, libellé_produit, pu, qté_commandée, ville, quartier, rue, Nom_client

Pour mettre une relation en deuxième forme normale, il faut l'éclater pour la débarrasser des augmentations dans la dépendance fonctionnelle entre la clé primaire et les autres attributs.
Pour ce faire, il faut prendre chaque attribut en but de la dépendance fonctionnelle et rechercher la partie de la source qui le détermine. Dans la DF numéro_bon, référence_produit →Numéro_client, il n'y a que l'attribut numéro_bon qui détermine le numéro du client. L'attribut référence_produit doit alors disparaitre de la source de la dépendance fonctionnelle. On fait la même chose pour tous les attributs en but et on obtient donc les dépendances fonctionnelles élémentaires suivantes :
numéro_bon →Numéro_client, date_commande, ville, quartier, rue, nom_client
référence_produit → libellé_produit, pu
numéro_bon, référence_produit → qté_commandée

Chaque DFE deviendra donc une relation dont la clé primaire est la source de la DFE et les attributs en but, les attributs de la relation.
R1 (**numéro_bon**, Numéro_client, date_commande, ville, quartier, rue, nom_client)
R2 (**référence_produit**, libellé_produit, pu)
R3 (**numéro_bon**, **référence_produit**, qté_commandée)

c - *La troisième forme normale*

Une relation est en 3ième forme normale si elle est en 2ième forme normale et si tous ses constituants sont en dépendance fonctionnelle élémentaire directe avec la clé primaire. C'est à dire, qu'aucune des dépendances fonctionnelles élémentaires de cette relation ne peut se déduire par transitivité ou encore que chaque attribut dépend directement de la clé primaire et non d'un autre attribut en but que la clé primaire.
Ex : La relation commander est - elle en 3ième forme normale ? Si non, mettez la en troisième forme normale.

Dans la DFE numéro_bon →Numéro_client, date_commande, ville, quartier, rue, nom_client on constate que des attributs en but ne dépendent pas uniquement de la clé primaire numéro_bon. En effet, il existe la DF Numéro_client → nom_client
Il faut donc éclater cette DFE qui deviendra
numéro_bon →Numéro_client, date_commande, ville, quartier, rue
Numéro_client → nom_client
Les autres dépendances fonctionnelles élémentaires n'ayant pas de transitivité, elles sont directes et restent telles. Notre relation Commander deviendra donc :

Bon_commande (**numéro_bon**, #Numéro_client, date_commande, ville, quartier, rue)
Client (**Numéro_client**, nom_client)
Produit (**référence_produit**, libellé_produit, pu)
Détail_commande (#**numéro_bon**, #**référence_produit**, qté_commandée)

Résumé

1ère forme normale	Dépendance fonctionnelle (DF)	Au plus une valeur
2ième forme normale	Dépendance fonctionnelle élémentaire (DFE)	Pas d'augmentation
3ième forme normale	Dépendance fonctionnelle élémentaire directe (DFED)	Pas de transitivité

Chapitre 3 : Le Modèle conceptuel des données ou modèle Entité-Association

Le modèle conceptuel des données est une représentation statique du système d'information de l'entreprise qui met en évidence sa sémantique. Il a pour but de décrire de façon formelle les données qui seront utilisées par le système d'information. Il s'agit donc d'une représentation des données, facilement compréhensible. Cet aspect recouvre les mots qui décrivent le système ainsi que les liens existants entre ces mots. Le formalisme adopté par la méthode Merise pour réaliser cette description est basé sur le concept « entité- association ».

I. Les concepts de base

a. La propriété (ou attribut ou rubrique)

La propriété est une information élémentaire, c'est-à-dire non déductible d'autres informations, qui présente un intérêt pour le domaine étudié. Il correspond à l'attribut en base de données relationnelle. *Par exemple, si l'on considère le domaine de gestion des commandes d'une société de vente par correspondance, les données : « référence article », « désignation article », « prix unitaire HT », « taux de TVA » sont des propriétés pertinentes pour ce domaine. La donnée « prix unitaire TTC » n'est, d'après la définition, pas une propriété car ses valeurs peuvent être retrouvées à partir des propriétés «prix unitaire HT » et « taux de TVA ».*

Chaque valeur prise par une propriété est appelée occurrence. *Des occurrences de la rubrique « désignation article » sont par exemple : « râteau », « bêche », « scie », …*

Une propriété est dite simple ou encore atomique si chacune des valeurs qu'elle regroupe n'est pas décomposable. *La propriété « Adresse », n'est pas élémentaire car elle peut être décomposée en trois propriétés : la rue, le code postal et la ville.*

b. L'entité ou individu ou objet

Définition

Une entité est la représentation d'un élément matériel ou immatériel ayant un rôle dans le système que l'on désire décrire. Chaque entité est composée de propriétés, données élémentaires permettant de la décrire.

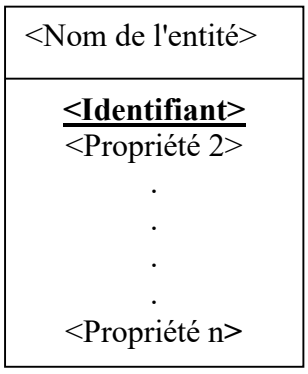
Exemple : Filière, Etudiant, Matière

Au premier abord, on peut définir l'entité comme étant un regroupement bien pensé, donc sensé, de plusieurs attributs. *Par exemple, on considère l'entité Etudiant qui regroupe les propriétés : Matricule, Nom, sexe et datedenaissance*. Le droit d'entrée d'une propriété dans une entité est soumis à d'autres facteurs que le bon sens, et ce sont ces facteurs que l'on va étudier.

L'entité est défini par :

- Une existence propre et une utilité pour le domaine d'étude
- Des occurrences multiples (au moins 2)
- Au moins une propriété

Pour représenter une entité, on utilise un rectangle de la forme :



Le nom de l'entité est un mot, généralement une abréviation pour une raison de simplification de l'écriture. Il s'agit par contre de vérifier qu'à chaque entité correspond un et un seul nom, et réciproquement.

c. Les identifiants

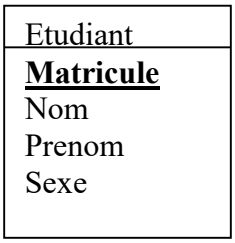
Un identifiant est un ensemble minimum d'attributs d'une entité dont la connaissance des valeurs permet d'identifier un enregistrement unique de l'entité considérée. Un identifiant est invariant dans le temps et est souligné lors de la représentation d'une entité.

Ainsi, pour une entité, l'attribut (ou les attributs) choisi comme identifiant ne peut avoir qu'une valeur unique. Deux occurrences de cette entité ne pourront jamais avoir la même valeur pour l'identifiant et donc, la connaissance d'une valeur de la rubrique identifiante détermine la connaissance des valeurs des autres rubriques de l'entité

Chaque entité doit posséder au moins un attribut identifiant, et l'ensemble de ses attributs identifiants doivent être renseignés à la création d'une occurrence de cette entité.

d. Occurrence d'entité ou d'individu

C'est un exemplaire de l'entité ou une ligne dans le tableau illustrant l'entité. Exemple, prenons l'entité Etudiant :

<table>
<tr><th>Représentation</th><th>Illustration</th></tr>
<tr><td>


</td><td>
<table>
<tr><th>Matricule</th><th>Nom</th><th>Prénom</th><th>Sexe</th></tr>
<tr><td>1</td><td>Magengo</td><td>Guttembert</td><td>M</td></tr>
<tr><td>2</td><td>Djangoni</td><td>Mandélai</td><td>F</td></tr>
<tr><td>3</td><td>Jolie</td><td>Joliette</td><td>F</td></tr>
<tr><td>4</td><td>Agogo</td><td>Bidas</td><td>M</td></tr>
</table>
« Magengo » est une occurrence de la propriété Nom
La ligne
<table>
<tr><td>1</td><td>Magengo</td><td>Guttembert</td><td>M</td></tr>
</table>
Est une occurrence de l'entité Etudiant
</td></tr>
</table>

e. L'association

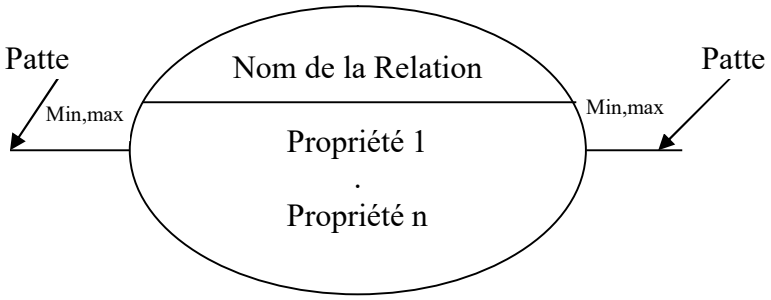
Définition

Une association est un lien sémantique entre plusieurs entités.

Voici les dénominations des différentes associations selon le nombre d'intervenants:

- Une association **récursive** (ou *réflexive*) relie la même entité
- Une association **binaire** relie deux entités
- Une association **ternaire** relie trois entités
- Une association **n-aire** relie n entités. Les associations sont représentées par des hexagones (parfois des ellipses) dont l'intitulé décrit le type de lien qui relie les entités (généralement un verbe).

Une association peut porter ou non des propriétés.



Elle est reliée au rectangle représentant les entités par des traits appelés pattes. Sur chacune des pattes de l'association, sont spécifiés les cardinalités minimum et maximum

f. Les cardinalités

Les cardinalités d'une entité dans une relation mesurent le nombre minimum et le nombre maximum de fois qu'une occurrence de cette entité participe à l'association.

Un couple de cardinalités placé entre une entité E et une association A représente le nombre minimal et maximal d'occurrences de l'association A qui peuvent être liées à une occurrence de l'entité E

On a généralement quatre types de cardinalités:

- 0,1 : (Au plus 1) l'occurrence de l'entité participe au plus une fois à l'association,
- 1,1 : (Un et un seul) l'occurrence de l'entité participe une et une seule fois à l'association.
- 1,N : (Au moins 1) l'occurrence de l'entité participe au moins une fois à l'association.
- 0,N : (Plusieurs) l'occurrence de l'entité participe plusieurs fois à la relation.

Remarque

La cardinalité est située sur la patte de l'association reliée à l'entité pour laquelle elle est définie.

La cardinalité minimum ne doit jamais être supérieur à 1

g. Dimension d'une association

C'est le nombre d'entités qui participent à l'association.

Association réflexive : dimension = 1

Association N-aire : dimension N

h. Contrainte d'intégrité fonctionnelle (cif) entre entités

Une contrainte d'intégrité fonctionnelle est une association établie entre deux entités et qui exprime que l'une de ces entités est totalement identifiée par la connaissance de l'autre. On l'appelle aussi association père-fils ou maitre-esclave.
Une contrainte d'intégrité fonctionnelle est toujours établie entre deux entités au plus et ne peut jamais porter d'attributs. Elle est matérialisée par la présence de la cardinalité maximum 1 sur au moins une des pattes (0,1 ou 1,1).
On peut remplacer le nom de l'association par le symbole **CIF**.

II. Règles de construction d'un M.C.D.

a. Les règles de gestion

Elles expriment les contraintes de gestion, de manipulation ou encore d'utilisation des données. Elles constituent le règlement intérieur dans le processus d'élaboration du MCD.
Elles précisent les liens qui peuvent être établis ou non entre les occurrences des entités ou entre les occurrences des attributs.
Exemple : Un étudiant s'inscrit dans une filière
Cette règle de gestion exprime un lien de dépendance fonctionnelle entre l'attribut Matricule et l'attribut CodeFilière et un lien d'association entre l'entité Etudiant et l'entité Filière.
Dans le processus de construction du MCD, il convient de bien recenser la liste exhaustive de ces règles de gestion.

b. Le Dictionnaire des données

C'est un tableau permettant d'inventorier toutes les informations manipulées par le domaine d'étude, qui ont un sens et qui peuvent avoir au moins deux occurrences. Ce tableau se présente comme suit :

Propriétés	Désignation en clair	Type	Nature	Longueur	Observation
Numat	Numéro matricule de l'étudiant	N	E	6	
NomEtu	Nom de l'étudiant	C	E	15	
DatNai	Date de naissance de l'étudiant	D	E	10	jj/mm/aaaa
SexEtu	Sexe de l'étudiant	C	E	1	« M » ou « F »

La colonne Propriété permet de spécifier pour chaque information, son nom. Le nom doit être court, explicite et doit être un mot. Il ne doit pas comporter de caractère interdits tels que : (,) (;) (-) (:) etc. mais peut comporter des combinaisons de lettres, de chiffres et le trait de 8(_) ou underscore

Désignation en clair permet de donner une description explicite de la propriété

Nature peut prendre les valeurs suivantes :

- E : élémentaire qui signifie que la propriété est atomique c'est-à-dire ni calculée, ni concaténée, ni décomposable.
- Co : Concaténée qui signifie que la propriété est décomposable. Dans ce cas, la formule de décomposition doit être spécifiée dans la colonne Observation et les informations variables entrant dans la décomposition doivent être ajoutés au dictionnaire des données.

- Ca : calculé qui signifie que la valeur de la propriété est issue d'un traitement ou de l'évaluation d'une formule avec des opérateurs numériques. Dans ce cas, la formule doit être spécifiée dans la colonne Observation et les informations variables qu'elle utilise doivent être ajoutées au dictionnaire des données.

Le type est le domaine de valeurs dans lequel une propriété peut puiser ses occurrences. On distingue les types suivants : Numérique N, caractère C, Logique L, Date D

La longueur est le nombre maximum de caractères qu'il faut pour représenter une occurrence de la propriété

La colonne Observation permet de faire tout commentaire qui pourrait rendre la propriété plus explicite.

c. La couverture minimale

C'est la liste de toutes les dépendances fonctionnelles élémentaires et directes c'est-à-dire des dépendances fonctionnelles débarrassées de toute augmentation et transitivité.
On distingue trois catégories de dépendance fonctionnelles :

- **Les dépendances fonctionnelles simples**

Ce sont les dépendances fonctionnelles élémentaires directes à sources non composées et dont le but ne contient pas d'identifiants
Exemple : Matricule → Nom_etu, sexe, Dat_nais

- **Les dépendances fonctionnelles à sources composées**

Ce sont les dépendances fonctionnelles élémentaires directes dont la source est composée d'au moins deux attributs en source et des attributs non identifiants en but.
Exemple : Code_matière, code_filière→Coefficient

- **Les dépendances fonctionnelles entre un identifiant en source et un identifiant en but**

Ce sont les dépendances fonctionnelles dans lesquelles la source est composée d'un identifiant et aussi le but, d'un identifiant.
Exemple : Un étudiant s'inscrit dans une et une seule filière : Matricule →Code_filière

d. La matrice des dépendances fonctionnelles

C'est un autre moyen de représenter la liste des dépendances fonctionnelles élémentaires directes. Mais, cette fois ci à partir d'un tableau à double entrée dans lequel on représente en en-tête de colonne, la liste de tous les identifiants (représentants les sources de dépendance fonctionnelles) et en en-tête de ligne, la liste de toutes les propriétés identifiants et non identifiants.
La matérialisation d'une DFED se fait en inscrivant le caractère étoile ou 1 à l'intersection de la source (entête de colonne) et du But (Entête de ligne)

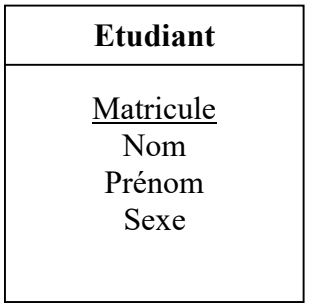
III. Construction du MCD à partir de la couverture minimale

La construction du MCD se fait à partir de la couverture minimale. Chacune des trois catégories de DFED correspond à une représentation particulière au niveau du MCD.

- La première catégorie de DFED (Les DFED simples) permet de représenter toutes les entités du MCD. En effet, toute dépendance fonctionnelle élémentaire directe entre une source non composée

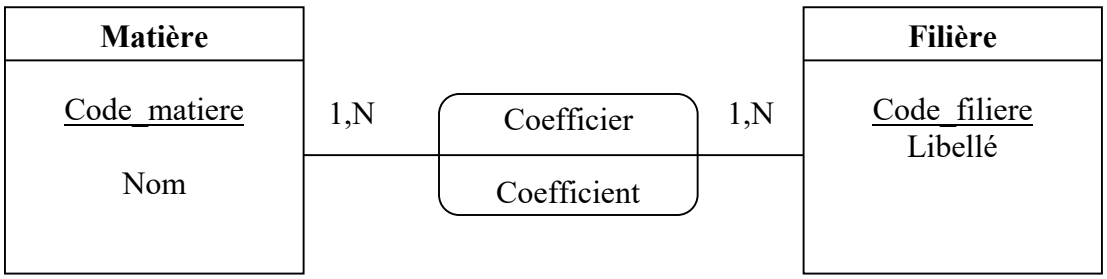
et des attributs non identifiants devient une entité dont l'identifiant est la source de la dépendance fonctionnelle élémentaire directe et dont les propriétés sont les attributs en but de la dépendance fonctionnelle.

Exemple : la dépendance fonctionnelle Matricule →Nom, prénom, sexe donnera l'entité Etudiant suivante :

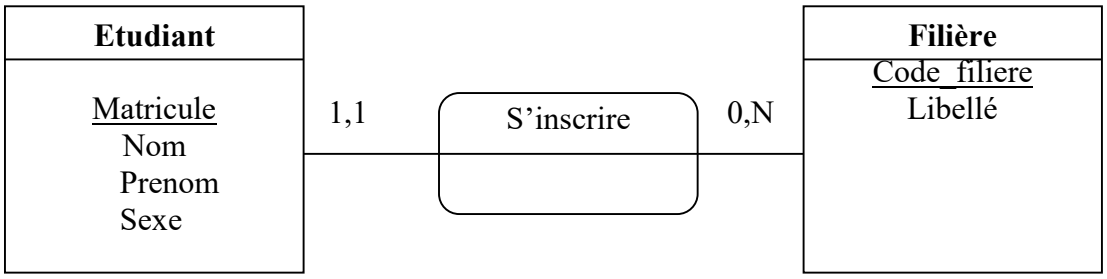


- La deuxième catégorie de DFED (Les DFED simples) permet de représenter toutes les associations plusieurs à plusieurs porteuses de propriétés du MCD. En effet, toute dépendance fonctionnelle élémentaire directe dont la source est composée devient une association entre les entités des identifiants qui composent la source. La cardinalité maximum sur toutes les pattes de l'association est N et les attributs en but de la dépendance fonctionnelle élémentaire directe deviennent les attributs de l'association.

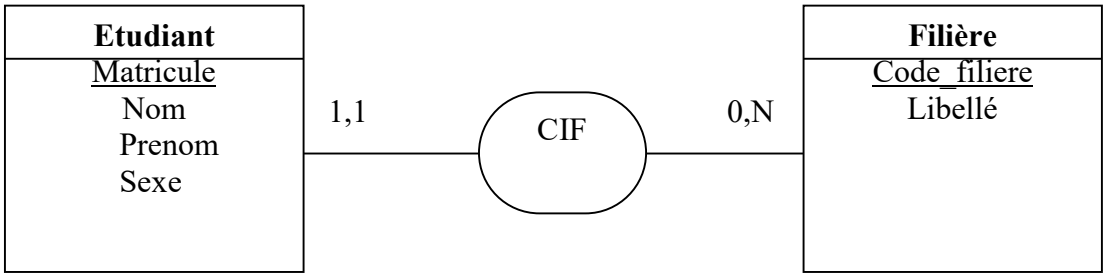
Exemple : la dépendance fonctionnelle Code_filiere, Code_matiere → coefficient donnera l'association suivante :



- La troisième catégorie de DFED (Les DFED entre identifiant et identifiant) permet de représenter toutes les associations père fils (CIF) du MCD. En effet, toute dépendance fonctionnelle élémentaire directe entre un identifiant et un identifiant devient une association CIF (Contrainte d'intégrité fonctionnelle) avec la cardinalité maximale 1 sur la patte de l'entité de l'identifiant en source de dépendance fonctionnelle.



Une telle association ne porte jamais de propriétés et elle peut être remplacée par le symbole CIF



Exercice d'application

Voir planche de Base de données. Exercice : gestion commerciale.

TAF

- Etablir le dictionnaire des données
- Présenter la couverture minimale
- Présenter la matrice des DFED
- Présenter le MCD

IV. Modélisation avancée

a. Les associations réflexives

Une association réflexive est une association qui fait intervenir au moins deux fois la même entité.

Elle peut être une association plusieurs à plusieurs. Dans ce cas, elle est représentée dans la couverture minimale par la DFED suivante : Id1,Id1→at1,…..,atN

L'identifiant de l'entité apparait deux fois en source de la DFED.

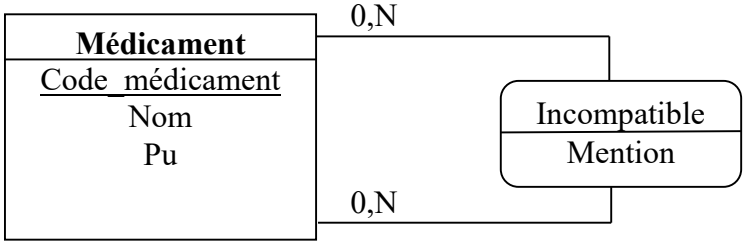
Exemple : Dans une entreprise pharmaceutique, on désire modéliser l'incompatibilité entre les médicaments. Un médicament peut être incompatible avec plusieurs autres médicaments. Pour chaque incompatibilité, une mention est définie : grave, moins grave, inoffensive etc. Un médicament a un code, un nom et un pu.

On aura la couverture minimale suivante :

Code_médicament→Nom, Pu

Code_médicament, Code_médicament →Mention

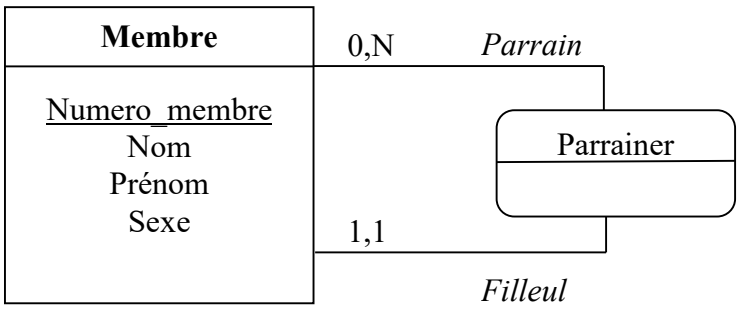
Ce qui donnera le MCD suivant :



Cette association peut aussi être une association père fils. Dans ce cas, elle est représentée dans la couverture minimale par la DFED suivante :

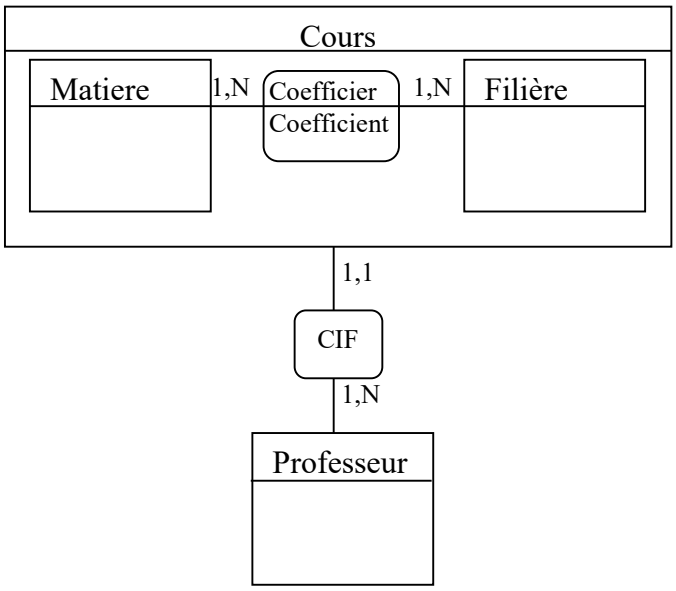
Id1 → Id1

Exemple : On considère une association dans laquelle chaque membre est parrainé par un autre membre. Sachant que un membre est décrit par son numéro, son nom, son prénom et son sexe et ne peut être parrainé que par un seul membre. Tandis qu'un membre peut parrainer plusieurs filleuls.
Numero_membre→Nom, prenom, sexe
Numero_membre →Numero_membre



b. Les pseudo entités

Toute dépendance fonctionnelle élémentaire directe entre une source composée et un identifiant devient une contrainte d'intégrité fonctionnelle entre une pseudo entité matérialisé par une association entre les entités des identifiants en source de la DFED et l'entité de l'identifiant en but de la dépendance fonctionnelle.
Exemple soit les DFED
Code_filiere, Code_matiere →Num_prof
Code_filiere, Code_matiere →coefficient
qui signifient que une matière dans une filière n'a qu'un coefficient et est enseignée par un et un seul professeur



Chapitre III - Algèbre relationnelle

L'algèbre relationnelle est une collection d'opérations permettant d'opérer sur les concepts du modèle relationnel. Elle permet par exemple de sélectionner certains enregistrements d'une relation satisfaisant une condition ou encore de regrouper des enregistrements de relations différentes. Le résultat de toute opération de l'algèbre est une nouvelle relation. Cette propriété implique notamment qu'il n'y a pas de doublons dans le résultat et permet l'écriture d'expressions de calcul. Etant donnée, que le modèle relationnel est basé sur la théorie des ensembles, l'algèbre relationnelle utilise les opérateurs classiques de manipulation des ensembles (union, intersection, différence et produit cartésien) et introduit des opérateurs propres aux bases de données (sélection, projection, jointure, division).

a – <u>les opérations portant sur une seule relation</u>

- ***<u>la projection</u>***

Elle consiste à construire une nouvelle relation R' à partir d'une relation R en ne retenant que certains attributs de R. les tuples(enregistrements ou articles) en double sont éliminés.

Syntaxe

PROJECTION.<nom relation>(<attribut1>,…,<attribut n>)

Exemple soit la relation R (refprod, desipro, pu, qtes) présentée comme la table ci-dessous :

R			
refpro	desipro	qtes	pu
AC	Akassa congelé	10	250
PP	Poisson pourri	5	400
RB	Riz brûlé	5	275

R'=PROJECTION.R (refpro, desipro)

La relation R' ainsi créée ne contiendra que les champs refpro et desipro

R	
refpro	desipro
AC	Akassa congelé
PP	Poisson pourri
RB	Riz brûlé

- ***<u>La sélection</u>***

Elle consiste à créer une nouvelle relation R' à partir d'une relation R en ne conservant que les tuples répondant à une condition donnée. La relation R' ne sera donc constituée que de tuples ne répondant qu'aux critères de sélection.

Syntaxe

SELECTION.< nom relation>(<critère de sélection>)

Ex : soit la relation R de l'exemple précédent

R' est la relation contenant tous les produits dont le pu est inférieur à 400
R'=SELECTION.R (Pu<400)

R'			
refpro	**desipro**	**qtes**	**pu**
AC	Akassa congelé	10	250
RB	Riz brûlé	5	275

On peut limiter les champs à afficher. Pour cela, on fait une projection sur R'.
Ex : afficher la référence et la quantité de tous les produits dont la qté égale 5

- ***Le tri***

Cette opération consiste à créer une nouvelle relation R' à partir d'une relation R en classant les tuples par ordre alphabétique ou numérique croissant ou décroissant sur un ou plusieurs champs.

Syntaxe

TRI.< nom relation>(<Attribut 1> [CROISSANT|DECROISSANT], …, <Attribut N> [CROISSANT|DECROISSANT])

Ex : soit la relation R de l'exemple précédent
R' est la relation contenant tous les produits classés par ordre croissant des références
R'=TRI.R (Refpro **CROISSANT**)

R'			
refpro	desipro	qtes	pu
AC	Akassa congelé	10	250
PP	Poisson pourri	5	400
RB	Riz brûlé	5	275

b - <u>les opérations portant sur des relations de même schéma</u>

- ***L'union***

Cette opération s'opérant sur deux relations consiste à ajouter à la première relation les tuples de la 2ième relation qui ne sont identiques à aucun tuple de la première.

Syntaxe

R1 U R2
Ex : soient les relations R1 et R2 de même schéma tel que R1=R2(refpro, desipro, pu)

R1		
refpro	desipro	pu
AC	Akassa congelé	250
PP	Poisson pourri	400
RB	Riz brûlé	275

R2		
refpro	desipro	pu
AC	Akassa congelé	250
PP	Poisson pourri	400
PV	Piment vert	275
ALS	Aloko sec	375

Trouver R tel que R =R1 U R2

- ***L'intersection***

L'intersection de deux relations r1 et r2 de même schéma est une relation R3 contenant les tuples appartenant à la fois à R1 et à R2. il s'agira donc de sélectionner les tuples communs aux deux relations.

Syntaxe

INTERSECTION (R1, R2) ou R1 ∩ R2

Ex :

- ***La différence***

Cette opération permet de créer une relation R3 à partir de deux relations R1 et R2 de même schéma contenant des tuples appartenant à R1 et n'appartenant pas à R2.

Syntaxe

DIFFERENCE (R1, R2) ou R1-R2

l'ordre de la relation est très important. R1-R2 est différent de R2-R1

c – Les opérations portant sur des relations de schémas différents

- ***le produit ou produit cartésien***

Cette opération permet d'obtenir toutes les relations possibles de deux relations de schéma différent.

Syntaxe

PRODUIT (R1, R2) ou R1 x R2

Ex :

- ***La jointure***

La jointure de R1 et R2 selon une condition est l'ensemble des tuples du produit cartésien R1xR2 qui satisfont à la condition indiquée. Cette opération permet donc de créer un sous-ensemble du produit cartésien R1xR2 qui satisfait à une condition portant sur un attribut de chaque relation. Le cas le plus indiqué est la jointure naturelle appelée jointure Equi-jointure fondée sur l'égalité des valeurs d'un attribut commun aux deux relations. Un seul des deux attributs de même valeur est conservé dans la relation obtenue.

Syntaxe

JOINTURE (R1, R2, <condition>)

Ex

- ***La division***

C'est le quotient de la relation R de schéma R (a1, a2, … , an) par la sous relation S de schéma S (Ap+1, … , An) qui est une relation Q de schéma Q (a1, a2, … , a_q) formé de tous les tuples qui concaténés à chacun des tuples de S donne toujours un tuple de R.

Syntaxe

DIVISION (R, S) ou **R / S** ou **R : S**

Ex

Soit les relations Composer et Matière telles que :

Composer			
Matricule	Date compo	Cod_matiere	Note
001	05/02/95	Ang	10
002	03/04/99	Ang	5
003	03/04/99	FR	15
001	03/04/99	FR	2
002	05/02/95	BD	18
001	05/02/95	BD	13

Matière	
Code_matiere	Libellé
Ang	Anglais
FR	Français
BD	Base de données

On désire afficher les matricules des étudiants qui ont composé dans toutes les matières

D = Projection.Composer (Matricule, Cod_matiere)

d = Projection.Matière (Cod_matiere)

Q = Division (D,d)

d – Les expressions d'attribut et le regroupement (Agrégation)

- **Les expressions d'attribut**

  Une expression d'attribut est une combinaison d'attributs, d'opérateurs numériques et de constante pouvant être assimilée à une formule qui, évaluée donne un résultat ou une information non prévue dans la relation.

  Exemple soit la relation R (refprod, desipro, pu, qtes) présentée comme la table ci-dessous :

R			
refpro	desipro	qtes	pu
AC	Akassa congelé	10	250
PP	Poisson pourri	5	400
RB	Riz brûlé	5	275

On désire afficher pour chaque produit, sa référence et son montant

R'=PROJECTION.R (refpro, Pu*Qtes)

R	
refpro	Pu*Qtes
AC	2500
PP	2000
RB	1375

Remarque : On peut changer le nom de la colonne contenant l'expression d'attribut en utilisant la clause COMME.

R'=PROJECTION.R (refpro, Pu*Qtes COMME Montant)

R	
refpro	Montant
AC	2500
PP	2000
RB	1375

- Le regroupement
  Le regroupement est une opération qui permet à partir d'une relation R de créer une relation R' contenant la synthèse des enregistrements de R.
  Syntaxe
  R'=REGROUPEMENT (R ; <Liste des attributs de regroupement> ; <Liste des fonctions de regroupement>)
  Les attributs de regroupement sont les attributs qui serviront à regrouper les enregistrements
  Exemple :
  Soit la table Etudiant (Matricule, nom, Sexe, Codfil) illustrée par le tableau suivant :

Matricule	**Nom**	Sexe	Codfil
001	Magengo	M	IIR
002	Jolie	F	SB
003	Marteau	M	BF
004	Cailloux	F	IIR
005	Gravière	F	IIR
006	BLANC	M	CG
007	MAHIGÔH	F	IIR
008	Djangoni	F	CG

On désire afficher le nombre d'étudiants par filière : les étudiants seront donc regroupés par filière afin qu'on puisse les compter. Le champ de regroupement est donc Codfil
Si On désire afficher le nombre d'étudiants par filière et par sexe, les étudiants seront donc regroupés par filière et par sexe afin qu'on puisse les compter. Les champs de regroupement seront donc Codfil et Sexe.
Une fonction de regroupement est une fonction prédéfinie qu'on applique sur un groupe d'enregistrement pour en obtenir la synthèse c'est-à-dire pour obtenir un enregistrement. En somme, les attributs de regroupement permettent de créer des groupes d'enregistrement sur lesquels on applique une fonction de regroupement pour obtenir la synthèse de chaque groupe. On distingue différentes fonctions de regroupement :
Compte(*) ou compte (<attribut>) : cette fonction permet de compter le nombre d'enregistrements dans un groupe
Somme (<attribut>) : cette fonction permet de faire la somme des valeurs de l'attribut spécifié d'un groupe d'enregistrement.
Moyenne (<attribut>) : cette fonction permet de faire la moyenne arithmétique des valeurs de l'attribut spécifié d'un groupe d'enregistrement.
Min / Max (<attribut>) : cette fonction permet de rechercher la plus petite ou la plus grande valeur de l'attribut spécifié d'un groupe d'enregistrement
Premier / Dernier (<attribut>) : cette fonction permet de rechercher la première ou la dernière valeur de l'attribut spécifié d'un groupe d'enregistrement

Exemple :
Soit la table Etudiant (Matricule, nom, Sexe, Codfil) illustrée par le tableau suivant :

Matricule	**Nom**	Sexe	Codfil

001	Magengo	M	IIR
002	Jolie	F	SB
003	Marteau	M	BF
004	Cailloux	F	IIR
005	Gravière	F	IIR
006	BLANC	M	CG
007	MAHIGÔH	F	IIR
008	Djangoni	F	CG

Afficher le nombre d'étudiants par filière
R=REGROUPEMENT (Etudiant ; Codfil ; Compte(*) comme Nombre)

Codfil	Nombre
IIR	4
SB	1
BF	1
CG	2

Afficher le nombre d'étudiants par filière et par sexe
R=REGROUPEMENT (Etudiant ; Codfil, Sexe ; Compte(*) comme Nombre)

Codfil	Sexe	Nombre
IIR	M	1
IIR	F	3
SB	F	1
BF	M	1
CG	F	1
CG	M	1

Remarque
Le résultat d'un regroupement est une relation contenant seulement les attributs de regroupement et les fonctions de regroupement. L'opération qui suit un regroupement ne peut donc utiliser que les attributs de regroupement et les fonctions de regroupement.

LE LANGAGE SQL

Le langage SQL(Structured querry language) vient des travaux de recherche menés chez IBM. C'est un langage de requête et un langage de description des données qui est utilisé directement sur un terminal grace à l'utilitaire interactif SQL ou dans un langage hote (Cobol, C, DbaseIV, Access ect…)

DEFINITIONS

Requête

Question sur les données enregistrées dans vos tables ou demande d'exécution d'une action sur les données. Une requête permet de réunir les données de plusieurs tables en vue de leur utilisation comme source de données pour divers traitements
Vous pouvez utiliser des requêtes pour afficher, modifier et analyser des données de diverses façons.

Requêtes de définition des données

Requête propre à SQL qui contient des instructions DDL (Data Definition Language). Celles-ci vous permettent de créer ou de modifier la structure des tables de la base de données.

Requêtes de manipulation des données

Contrairement aux Requêtes de définition des données, les Requêtes de manipulation des données permettent d'agir sur les données des tables et non sur leurs structures.

II - LES INSTRUCTIONS DE REQUETES DE DEFINITION DES DONNEES

Utilisez ces instructions pour créer des requêtes Définition des données SQL qui créent, suppriment ou modifient des tables.

- **Instruction CREATE TABLE simplement**

Crée une nouvelle table.

Syntaxe

CREATE TABLE <*table*> (<*champ1*> < *type*> [(<*taille*>)] [<Contrainte de champ>] ,….., <*champN*> < *type*> [(<*taille*>)] [<Contrainte de champ>],[<Contrainte de table>])

L'instruction CREATE TABLE se compose des éléments suivants :

Élément	Description
table	Nom de la table à créer.
champ1, *champ2*	Nom du ou des champs à créer dans la nouvelle table. Vous devez créer au moins un champ.
type	Type de données du *champ* dans la nouvelle table.
taille	Taille du champ en caractères (pour les données de type Texte ou

	Binaire uniquement).

[<Contrainte de champ>] vous permet d'appliquer à un champ l'un des types de contraintes suivants :

- ***NOT NULL*** : Vous pouvez utiliser ces mots réservés pour spécifier qu'un champ ne peut contenir ne peut rester vide. Cela signifie qu'un enregistrement de la table ne peut être validé que si ce champ contient une valeur. Vous pouvez contraindre n'importe quel champ comme étant non vide.
CREATE TABLE Filière (Codfil INTEGER NOT NULL, Libellé CHAR (50) NOT NULL)

- ***UNIQUE*** : Vous pouvez utiliser ce mot réservé pour désigner un champ en tant que clé unique. Cela signifie que deux enregistrements de la table ne peuvent pas avoir la même valeur dans ce champ. Vous pouvez contraindre n'importe quel champ ou n'importe quelle liste de champs comme étant unique.
CREATE TABLE Filière (Codfil INTEGER ***UNIQUE***, Libellé CHAR (50) NOT NULL)

- ***PRIMARY KEY*** : Vous pouvez utiliser ces mots réservés pour désigner un champ ou ensemble de champs dans une table en tant que clé primaire. Toutes les valeurs de la clé primaire doivent être uniques, non **Null**, et il ne peut y avoir qu'une clé primaire par table. Ne définissez pas de contrainte PRIMARY KEY sur une table qui possède déjà une clé primaire. Une erreur se produira si vous le faites.
CREATE TABLE Filière (Codfil INT PRIMARY KEY, Libellé CHAR (50))

- ***REFERENCES table [déclencheur {RESTRICT | NO ACTION | SET NULL | SET DEFAULF | CASCADE}*** et Contrainte d'intégrité référentielle
Vous pouvez utiliser ce mot réservé pour désigner un champ en tant que clé étrangère. Si le ou les champs référencés sont la clé primaire de la table étrangère, vous n'avez pas besoin de spécifier les champs référencés. Par défaut, le moteur de base de données se comporte comme si la clé primaire de la table étrangère était les champs référencés.
CREATE TABLE Etudiant (Matricule INTEGER PRIMARY KEY, Nom CHAR (255), Codfil INTEGER REFERENCES Filière)

Les déclencheurs sont ON DELETE et ON UPDATE. Ils sont suivis des arguments entre accolades permettant de spécifier l'action à réaliser en cas d'effacement d'une ligne de la table faisant partie de la clé étrangère:

- CASCADE indique la suppression ou la modification en cascade des lignes de la table étrangère dont les clés étrangères correspondent aux clés primaires des lignes effacées
- RESTRICT ou NO ACTION est le comportement par défaut. Si l'on essaye de supprimer une valeur référencée par une clé étrangère, l'action est avortée et on obtient une erreur. NO ACTION a exactement le même effet.
- SET NULL place la valeur NULL dans la ligne de la table étrangère en cas d'effacement d'une valeur correspondant à la clé
- SET DEFAULT place la valeur par défaut dans la ligne de la table étrangère en cas d'effacement d'une valeur correspondant à la clé

DEFAULT *valeur*

CHECK *condition*

Les contraintes CHECK assurent l'intégrité de domaine en limitant les valeurs acceptées par une ou plusieurs colonnes. Vous pouvez créer une contrainte CHECK avec n'importe quelle expression logique (booléenne) qui retourne TRUE (vrai) ou FALSE (Faux) sur la base des opérateurs logiques.

Par exemple, il est possible de limiter la plage de valeurs d'une colonne **Sexe** en créant une contrainte CHECK qui autorise uniquement les données M et F. De cette façon, il est impossible d'entrer une valeur erronnée comme G pour dire Garçon. L'expression logique est la suivante : `Sexe='M' Or Sexe='F'.`

Vous pouvez appliquer plusieurs contraintes CHECK à une seule colonne (Contrainte d'attribut). Vous pouvez aussi appliquer une seule contrainte CHECK à plusieurs colonnes en la créant au niveau des contraintes de table. Ainsi, vous pouvez utiliser une contrainte CHECK sur plusieurs colonnes pour confirmer que les lignes comportant la valeur **Marié** dans leur colonne **Situation matrimoniale** et la valeur **F** dans leur colonne Sexe possèdent forcément une valeur dans leur colonne **Nom de jeune fille**. Cela permet de vérifier plusieurs conditions au même emplacement.

Les contraites CHECK sont similaires aux contraintes FOREIGN KEY dans la mesure où elles contrôlent les valeurs qui sont placées dans une colonne. Leur différence réside dans la manière dont elles déterminent les valeurs considérées comme valides : les contraintes FOREIGN KEY obtiennent la liste des valeurs valides d'une autre table, tandis que les contraintes CHECK déterminent ces valeurs sur la base d'une expression logique.

[<Contrainte de table>] vous permet de spécifier des contraintes liées à l'ensemble de la table comme une contrainte de clé primaire composée de plusieurs attributs :

Syntaxe :

PRIMARY KEY (<champ1>,…….,<champN>)

Exemple :

CREATE TABLE Coefficier(Codmat int REFERENCES Matière, codfil int REFERENCES Filière, coefficient int, ***PRIMARY KEY*** (codmat,codfil))

Ou une contrainte de domaine composée de plusieurs attributs

CREATE TABLE Employé (Matricule int *PRIMARY KEY* , Nom char, Nom_jeune_fille char, sexe char, situation_matrimoniale char, CHECK (Sexe='F' and situation_matrimoniale='Marié' and Nom_jeune_fille <>Null)

Les Types de données

Type de données	Description
BIT	Valeurs Yes et No ainsi que les champs qui ne peuvent contenir qu'une ou deux valeurs.
MONEY	Un entier scalaire compris entre – 922 337 203 685 477,5808 et 922 337 203 685 477,5807.
DATETIME	Une valeur date ou heure comprise entre l'année 100 et l'année 9999.
REAL	Une valeur en virgule flottante à simple précision, comprise entre – 3,402823E38 et – 1,401298E-45 pour les valeurs négatives et entre 1,401298E-45 et 3,402823E38 pour les valeurs positives, et 0.

SMALLINT	Un entier court compris entre – 32 768 et 32 767 (voir Remarques).
INTEGER	Un entier long compris entre – 2 147 483 648 et 2 147 483,647 (voir Remarques).
DECIMAL	Type de données numérique exact contenant des valeurs comprises entre 10^{28} - 1 et - 10^{28} - 1. Vous pouvez définir la précision (1 - 28) ainsi que l'échelle (0 - précision définie). La précision et l'échelle par défaut sont 18 et 0 respectivement.
TEXT	De zéro à 2.14 giga-octets au maximum.
CHAR	De zéro à 255 caractères.

- **Instruction ALTER TABLE**

Modifie la structure d'une table.

Syntaxe

- ***Ajouter un champ à une table***

ALTER TABLE <*table*> ADD COLUMN <*champ*><*type*> [(<*taille*>)] [<Contrainte>]

- ***Modifier le type, la taille ou la contrainte d'un champ d'une table***

ALTER TABLE <*table*> ALTER COLUMN <*champ*><*type*> [(<*taille*>)] [<Contrainte>]

- ***Supprimer le champ d'une table***

ALTER TABLE <*table*> DROP COLUMN <*champ*>

L'instruction ALTER TABLE se compose des éléments suivants :

Élément	**Description**
table	Nom de la table à modifier.
champ	Nom du champ à ajouter ou à supprimer dans la *table*. Ou nom du champ à modifier dans la *table*.
type	Type de donnée du *champ*.
taille	Taille du champ en caractères (pour les données de type Text ou Char uniquement).

Notes

L'instruction ALTER TABLE permet de modifier une table existante de plusieurs façons. Pour ce faire, vous pouvez :

- Utiliser ADD COLUMN pour ajouter un nouveau champ à la table. Spécifiez alors le nom du champ, son type de données et (dans le cas de champs Text et Char) sa taille (facultative). Par exemple, l'instruction suivante ajoute un champ de texte de 25 caractères appelé Notes (Remarques) à la table Employés (Employés) :

  ALTER TABLE Employés ADD COLUMN Notes TEXT(25)

  Si vous spécifiez l'indication NOT NULL pour un champ, les nouveaux enregistrements doivent contenir des données valides dans ce champ.

- Utilisez ALTER COLUMN pour changer le type de données d'un champ existant. Spécifiez le nom du champ, le nouveau type de données et (dans le cas de champs Text et Char) sa taille (facultative). Par exemple, l'instruction suivante modifie le type de données du champ matricule de la table Employés (Employés) (initialement défini comme Entier) en un champ de type Char de 10 caractères :

  ALTER TABLE Employés ALTER COLUMN matricule CHAR(10)

- Utiliser DROP COLUMN pour supprimer un champ. Spécifiez alors uniquement le nom du champ.

Remarques

- Vous ne pouvez pas ajouter, supprimer ou modifier plusieurs champs à la fois.

- **Instruction DROP**

Supprime une table d'une base de données.

Syntaxe

DROP TABLE <*table*>

L'instruction DROP se compose des éléments suivants :

Élément	**Description**
table	Nom de la table à supprimer.

Notes
Vous devez fermer la table avant de la supprimer.

IV - INSTRUCTIONS DML

- **Instruction INSERT INTO**

Ajoute un ou plusieurs enregistrements à une table. C'est ce qu'on appelle une ***requête Ajout***.

Syntaxe

Requête Ajout avec plusieurs enregistrements :

Voir instruction SELECT

Requête Ajout avec un seul enregistrement :

INSERT INTO <nom de la table> [(<*champ1*>,…, <*champN*>)] VALUES (<*valeur1*>,…,< *valeurN*>)

L'instruction INSERT INTO comprend les éléments suivants :

Élément	Description
nom de la table	Nom de la table à laquelle on souhaite ajouter des enregistrements.
champ1, *champN*	Noms des champs auxquels on souhaite ajouter les données.
valeur1, *valeurN*	Valeurs à insérer dans des champs spécifiques du nouvel enregistrement. Chaque valeur vient s'insérer dans le champ correspondant à la position de la valeur dans la liste : *valeur1* s'insère dans le *champ1* du nouvel enregistrement, *valeur2* dans le *champ2* et ainsi de suite. Vous devez séparer les valeurs par une virgule, encadrer les chaines de caractère littérale par des guillemets simples (' ') ou des griffes (" ").

Notes

Si vous ne spécifiez pas tous les champs, la valeur par défaut ou ***Null*** s'insère à la place des colonnes manquantes. Les enregistrements sont ajoutés à la fin de la table.

.

- **Instruction UPDATE**

Crée une ***requête de mise à jour*** qui modifie les valeurs des champs d'une table spécifiée, selon des critères déterminés.

Syntaxe

UPDATE <*table*>
SET <champ1>=<*nouvellevaleur1*>[,…….,<champN>=<*nouvellevaleurN*>]
[WHERE <*critère*>];

L'instruction UPDATE comprend les éléments suivants :

Élément	Description
Table	Nom de la table contenant les données que vous voulez modifier.
nouvellevaleur	Expression qui détermine la valeur à insérer dans un champ déterminé des enregistrements mis à jour.
critère	Expression qui détermine quels enregistrements seront mis à jour. Seuls les enregistrements correspondant à l'expression seront mis à jour.

Notes

Vous pouvez modifier simultanément plusieurs champs.

- **Instruction DELETE**

Crée une ***requête Suppression*** qui supprime des enregistrements dans une ou dans plusieurs des tables mentionnées dans la clause ***FROM*** qui correspond à la clause ***WHERE***.

Syntaxe

DELETE
FROM <*table*>
[WHERE <*critère*>]

L'instruction DELETE comprend les éléments suivants :

Élément	**Description**
table	Nom de la table dans laquelle des enregistrements sont supprimés.
critère	***Expression*** qui détermine quels sont les enregistrements à supprimer.

Notes

Vous pouvez utiliser DELETE pour supprimer des enregistrements impliqués dans une ***relation 1-n*** avec d'autres tables. Les opérations de ***suppression en cascade*** entraînent la suppression des enregistrements des tables placées du côté "plusieurs" de la relation lorsque l'enregistrement correspondant, placé du coté "un" de la relation, est lui-même supprimé de la requête. Par exemple, dans la relation établie entre les tables Clients et Commandes, la table Clients se situe du côté "un" tandis que la table Commandes est du côté "plusieurs". Supprimer un enregistrement dans Clients entraîne la suppression des enregistrements correspondants dans Commandes si l'option de suppression en cascade a été choisie.

Une requête de suppression efface des enregistrements dans leur intégralité et pas seulement les données de certains champs. Si vous désirez supprimer des valeurs contenues dans un champ déterminé, créez une ***requête de mise à jour*** pour remplacer les valeurs par ***Null***.

Important

- Une fois que vous avez supprimé des enregistrements à l'aide d'une requête de suppression ou de mise à jour, vous ne pouvez pas annuler l'opération. Si vous désirez savoir quels enregistrements seront supprimés ou modifiés, faites d'abord une ***requête Sélection*** avec les mêmes critères, examinez les résultats obtenus, puis exécutez la requête Suppression ou mise à jour.
- Conservez toujours des copies de sauvegarde de vos données. Si vous supprimez ou modifiez des enregistrements par inadvertance, vous pourrez les récupérer à partir de vos copies de sauvegarde.

- **Instruction SELECT**

Affiche des informations depuis la base de données sous la forme d'un jeu d'enregistrements.

Syntaxe

SELECT [<*étendue*>] <spécification des champs à afficher>
FROM <*liste des tables*>
[WHERE <Condition de jointure et condition de sélection avant regroupement>]
[GROUP BY <Liste des champs de regroupement>]
[HAVING <Condition de sélection après regroupement>]
[ORDER BY <champ1>[ASC ou DESC],…, <champN>[ASC ou DESC]
L'instruction SELECT se compose des éléments suivants :

Part	**Description**
étendue	L'un des prédicats suivants : ***ALL, DISTINCT, TOP***. Les prédicats permettent de limiter le nombre d'enregistrements renvoyés. Si aucun n'est précisé, ALL est choisi par défaut.
spécification des champs à afficher	Indique les champs qui seront sélectionnés dans le jeu de résultat.
liste des tables	Nom de la ou des tables contenant les données à utiliser dans la requête.

REMARQUE

On peut utiliser l'instruction SELECT pour ajouter un groupe d'enregistrements dans une table.

Syntaxe

Requête Ajout avec plusieurs enregistrements :

INSERT INTO <table *cible*> [(*champ1*[, *champ2*[, ...]])]
SELECT [*source*.]*champ1*[, *champ2*[, ...]
FROM *expressiontable*

L'instruction INSERT INTO comprend les éléments suivants :

Élément	**Description**
nom de la table	Nom de la table à laquelle on souhaite ajouter des enregistrements.
champ1, *champ2*	Noms des champs auxquels on souhaite ajouter les données, s'ils sont placés à la suite de l'argument *cible*, ou noms des champs à partir desquels on souhaite extraire les données, s'ils sont placés à la suite de l'argument *source*.
basededonnéesexterne	Chemin d'accès d'une ***base de données externe***. Pour obtenir une description du chemin, voir la clause ***IN***.
Source	Nom de la table ou de la requête à partir de laquelle on souhaite copier les enregistrements.
expressiontable	Nom de la ou des tables à partir desquelles les enregistrements sont insérés. Cet argument peut être constitué d'un seul nom de table, d'un nom de requête enregistrée ou d'un composé résultant d'une opération ***INNER JOIN, LEFT JOIN*** ou ***RIGHT JOIN***.
valeur1, *valeur2*	Valeurs à insérer dans des champs spécifiques du nouvel enregistrement. Chaque valeur vient s'insérer dans le champ correspondant à la position de la valeur dans la liste : *valeur1* s'insère dans le *champ1* du nouvel enregistrement, *valeur2* dans le *champ2* et ainsi de suite. Vous devez séparer les valeurs par une virgule et encadrer les zones de texte par des guillemets simples (' ').

- **Prédicats ALL, DISTINCT, TOP**

Spécifie les enregistrements sélectionnés par des requêtes ***SQL***.

Syntaxe

SELECT [ALL | DISTINCT | [TOP *n* [PERCENT]]]
FROM *table*

Une instruction ***SELECT*** contenant ces prédicats se compose des éléments suivants :

Élément	**Description**
ALL	Sélectionne tous les enregistrements qui remplissent les conditions de l' instruction SQL. Les deux exemples suivants sont équivalents et renvoient tous les enregistrements de la table Employés : SELECT ALL * FROM Employés;
DISTINCT	Omet tous les enregistrements pour lesquels les champs sélectionnés contiennent des

	données en double. Ainsi, pour être incluses dans les résultats de la requête, les valeurs de chaque champ répertorié dans l'instruction SELECT doivent être uniques. Par exemple, plusieurs employés répertoriés dans une table Employés (Employés) peuvent avoir le même nom. Si deux enregistrements contiennent Smith dans le champ Nom, l'instruction SQL suivante ne renvoie alors qu'un seul de ces enregistrements : SELECT DISTINCT Nom FROM Employés; Si vous omettez DISTINCT, cette requête renvoie les deux enregistrements Smith. Si la clause SELECT contient plusieurs champs, la combinaison des valeurs de tous ces champs doit être unique, pour un enregistrement donné, pour que ces valeurs apparaissent dans le résultat. Le résultat d'une requête qui utilise DISTINCT ne peut pas être mis à jour et ne reflète pas les modifications ultérieures effectuées par d'autres utilisateurs.
TOP *n* [PERCENT]	Renvoie un certain nombre d'enregistrements situés au début ou à la fin d'une plage d'enregistrement. Supposons que vous souhaitiez obtenir les noms des 25 premiers étudiants de la table étudiant : SELECT TOP 25 nom, prénom FROM étudiant; Si vous ne triez pas les enregistrements, la requête renverra une série de 25 enregistrements choisis arbitrairement parmi ceux de la table Etudiants qui remplissent les conditions de la clause WHERE si elle existe. Le prédicat TOP n'effectue pas de choix entre des valeurs égales. Dans l'exemple précédent, si, parmi les meilleurs résultats obtenus, le vingt-cinquième et le vingt-sixième ont obtenu la même moyenne, la requête renvoie 26 enregistrements. Vous pouvez également utiliser le mot réservé PERCENT pour renvoyer un certain pourcentage des premiers ou derniers enregistrements d'une plage d'enregistreent. Supposons qu'au lieu des 25 meilleurs étudiants, vous souhaitiez sélectionner 10 pour cent des étudiants: SELECT TOP 10 PERCENT nom, prénom FROM étudiant; La valeur qui suit TOP doit être un ***entier*** non signé. TOP n'affecte pas les possibilités de mise à jour de la requête.
table	Nom de la table contenant les enregistrements à extraire.

- **Spécification des champs à sélectionner**

Syntaxe

<spécification des champs à sélectionner>={ * | *table*.* | [*table*.]*champ1* [AS *alias1*] [, [*table*.]*champ2* [AS *alias2*] [, ...]]}

- utilisez l'astérisque pour sélectionner tous les champs de toutes les tables utilisées dans la requête.

- <table>.* permet de sélectionner tous les champs d'une table donnée.

- [<*table*>.]<*champ*> permet de lister les champs à sélectionner. Le nom de la table est facultatif. Il n'est nécessaire que lorsqu'un champ à sélectionner se retrouve dans deux tables utilisées dans la requête.

- [AS *alias1*] permet chaque fois que vous utilisez des ***fonctions d'agrégation*** ou des requêtes qui renvoient des noms d'attributs ou d'expressions d'attributs ambigus ou en double, de fournir un nom de remplacement assez explicite.

- **Clause FROM**

Spécifie les tables ou les requêtes qui contiennent les champs spécifiés dans l'instruction ***SELECT***.

Syntaxe

SELECT *listechamp*
FROM <*liste des tables*>

Une instruction SELECT contenant une clause FROM se compose des éléments suivants :

Élément	Description
Listechamp	Noms du ou des champs à extraire avec leurs ***alias*** de nom de champ, ***fonctions d'agrégation SQL***, prédicats de sélection (***ALL, DISTINCT, DISTINCTROW, ou TOP***) ou autres options d'instruction SELECT.
liste des tables	Expression qui identifie la ou les tables contenant les données à extraire. L'expression peut être constituée d'un seul nom de table, d'un nom de requête enregistrée ou d'un composé résultant d'une opération ***INNER JOIN***, ***LEFT JOIN*** ou ***RIGHT JOIN***.

Notes

Expressiontable peut être :

- Une table : dans ce cas, les données à utiliser dans cette requête proviennent toutes de cette table
- *Plusieurs tables : dans ce cas une jointure s'impose. Pour exprimer cette jointure, on peut :*

- *Lister les tables et spécifier les conditions de jointure dans la clause* ***where.*** *Exemple : afficher les nom et prénom des étudiants avec leur libellé de filière.*
- *Utiliser l'opération* ***INNER JOIN*** *qui Fusionne les enregistrements de deux tables lorsqu'un champ commun contient des valeurs identiques****.***

Syntaxe

FROM *table1* INNER JOIN *table2* ON *table1.champ1 oprcomp table2.champ2*

L'opération INNER JOIN comprend les éléments suivants :

Élément	Description
table1, *table2*	Noms des tables depuis lesquelles les enregistrements sont fusionnés.
champ1, *champ2*	Noms des champs reliés entre eux. S'ils ne sont pas numériques, les champs doivent avoir le même ***type de données*** et contenir des données de nature identique, mais leurs noms peuvent être différents.
Oprcomp	Tout opérateur de comparaison relationnelle tel que "=","<", ">", "<=", ">=", ou "<>". "=," "<," ">," "<=," ">=," ou "<>." Mais égal est le plus utilisé.

Pour avoir dans le jeu de résultats, toutes les occurrences de l'une des tables et seulement celles de l'autre table pour lesquelles les champs joints sont égaux, on effectue une jointure externe gauche avec l'opérateur LEFT JOIN ou une jointure externe droite avec l'opérateur RIGHT JOIN.

Une jointure LEFT JOIN ou RIGHT JOIN peut être imbriquée dans une jointure INNER JOIN mais il est impossible d'imbriquer une jointure INNER JOIN dans une jointure LEFT JOIN ou RIGHT JOIN

La clause FROM est requise et doit figurer à la suite de toute instruction SELECT.
L'ordre des noms de tables dans *expressiontable* est sans importance.

- **Clause WHERE**

Indique quels enregistrements des tables citées dans la clause ***FROM*** sont affectés par l'instruction ***SELECT*** **.**

Syntaxe

SELECT *listechamp*
FROM *expressiontable*
WHERE *critère*

Une instruction SELECT contenant une clause WHERE se compose des éléments suivants :

Élément	Description
Listechamp	Nom du ou des champs à extraire avec leurs ***alias*** de nom de champ, prédicats de sélection (***ALL, DISTINCT, DISTINCTROW, ou TOP***) ou autres options d'instruction SELECT.
expressiontable	Nom de la ou des tables contenant les données à extraire.
Critère	***Expression*** que les enregistrements doivent satisfaire pour être inclus dans le résultat de la requête.

Une clause WHERE peut contenir jusqu'à 40 expressions liées par des opérateurs logiques comme **And** et **Or**.
Lorsque vous entrez un nom de champ contenant un espace ou un signe de ponctuation, placez ce nom entre crochets ([ ]).
Si vous spécifiez l'argument *critère*, vous devez indiquer les ***littéraux date*** entre deux caractères #.
Lorsque vous utilisez des valeurs littérales de type caractère ou texte, encadrez les de griffes (")

LES OPERATEURS SQL

Quelques définitions

Expression

Toute combinaison d'opérateurs, de constantes, de valeurs littérales, de fonctions et de noms de champs (colonnes), de contrôles et de propriétés dont l'évaluation fournit une valeur unique. Vous pouvez utiliser les expressions en tant que paramètres pour un grand nombre de propriétés et d'arguments d'action ; pour définir des contrôles calculés dans des formulaires, des états et des pages d'accès aux données ; pour définir des critères (conditions de recherche) ou définir des champs calculés dans des requêtes ; et pour fixer des conditions dans des macros. Dans un projet Microsoft Access, les expressions vous permettent de définir des contraintes de validation.

Opérateur

Symbole ou mot, tel que > et Ou, qui indique l'opération à effectuer sur un ou plusieurs éléments. Microsoft Access possède plusieurs classes d'opérateurs, qui sont les opérateurs arithmétiques, logiques, de comparaison et de concaténation.

Critères

Ensemble de conditions limitatives, par exemple = « Danemark » (c'est-à-dire égal à Danemark) ou >30000, utilisées lors de la création d'une requête ou d'un filtre pour afficher un jeu spécifique d'enregistrements.

Les opérateurs

- **Opérateur Between...And**

Détermine si la valeur d'une expression est comprise dans un intervalle de valeurs donné. Vous pouvez utiliser cet opérateur à l'intérieur d'une instruction SQL

Syntaxe

<*Expression*> [**Not**] **Between** <*valeur1*> **And** <*valeur2*>

La syntaxe de l'opérateur **Between...And** comprend les éléments suivants :

Élément	Description
<*expression*>	Expression qui identifie le champ contenant les données que vous voulez évaluer.
valeur1, *valeur2*	Expressions auxquelles vous désirez confronter *expression*.

Remarques

Si la valeur de *expression* est comprise entre *valeur1*et *valeur2* (incluses), l'opérateur **Between...And** renvoie **True** ; sinon il renvoie **False**. Vous pouvez ajouter l'opérateur logique **Not** pour évaluer la condition contraire (autrement dit, savoir si *expression* se situe hors de l'intervalle défini par *valeur1*et *valeur2*).

- **Opérateur In**

Détermine si la valeur d'une expression est égale à l'une des valeurs comprises dans une liste donnée.

Syntaxe

<*expression*> [**Not**] **In(<***valeur1*>,< *valeur2*>, . . .,<*valeur n*>**)**

Remarques

La syntaxe de l'opérateur **In** comprend les éléments suivants :

Élément	**Description**
expression	Expression qui identifie le champ contenant les données que vous voulez évaluer.
valeur1, *valeur2*	Expression ou liste d'expressions que vous désirez confronter à <*expression*>.

Si on trouve <*expression*> dans la liste des valeurs, l'opérateur **In** renvoie la valeur **True**, dans le cas contraire, il renvoie **False**. Vous pouvez ajouter l'opérateur logique **Not** pour évaluer la condition contraire (autrement dit, savoir si <*expression*> ne figure pas dans la liste de valeurs communiquée.
Par exemple, vous pouvez utiliser l'opérateur **In** pour déterminer quelles commandes sont expédiées vers un ensemble de régions données.

```
SELECT *
FROM
WHERE [Région livraison] In ('zou','Collines','atlantique')
```

- **Opérateur Like**

Compare une expression chaîne avec un modèle dans une expression SQL.

Syntaxe

<*expression*> **Like** <*modèle*>

La syntaxe de l'opérateur **Like** comprend les éléments suivants :

Élément	**Description**
expression	Expression SQL utilisée dans une clause WHERE.
modèle	Chaîne ou chaîne de caractères constante à laquelle *expression* est comparée.

Remarques

Vous pouvez utiliser l'opérateur **Like** pour rechercher dans un champ des valeurs correspondant au modèle que vous spécifiez. Pour *modèle*, vous pouvez spécifier la valeur complète, par exemple, Like "Martin"), ou vous pouvez utiliser des caractères génériques pour rechercher un intervalle de valeurs (par exemple Like "Ma*").
Dans une expression, vous pouvez utiliser l'opérateur **Like** pour comparer une valeur de champ avec une expression de chaîne. Par exemple, si vous spécifiez Like "C*" dans une requête SQL, la requête renvoie toutes les valeurs de champ commençant par la lettre C. Dans une requête Paramètre, vous pouvez demander à l'utilisateur de préciser un modèle de chaîne à rechercher.
Dans l'exemple suivant, une requête renvoie les données commençant par la lettre P, suivie d'une lettre comprise entre A et F puis de trois chiffres :

```
Like "P[A-F]###"
```

Le tableau suivant vous montre comment utiliser **Like** pour évaluer des expressions avec différents modèles.

Type d'occurrence	**Modèle**	**Occurrence (renvoie True)**	**Aucune occurrence (renvoie False)**
Plusieurs caractères	a*a	aa, aBa, aBBBa	aBC
	* ab*	abc, AABB, Xab	aZb, bac
Caractère spécial	a[*]a	a*a	aaa
Plusieurs caractères	ab*	abcdefg, abc	cab, aab
Caractère isolé	a?a	aaa, a3a, aBa	aBBBa
Chiffre isolé	a#a	a0a, a1a, a2a	aaa, a10a
Intervalle de caractères	[a-z]	f, p, j	2, &
Hors d'un intervalle	[!a-z]	9, &, %	b, a
Autre qu'un chiffre	[!0-9]	A, a, &, ~	0, 1, 9

Combinaison	a[!b-m]#	An9, az0, a99	abc, aj0

Les caractères génériques
L'astérisque (*), le point d'interrogation (?), le signe dièse (#), le point d'exclamation (!), le trait d'union (-) et les crochets ([ ]) sont des caractères génériques. Vous pouvez utiliser ces caractères dans des requêtes et des expressions pour inclure tous les enregistrements, noms de fichier ou autres éléments dont l'intitulé commence par certains caractères ou correspond à une séquence particulière. Vous pouvez également utiliser les caractères génériques et les caractères correspondants pour affiner une recherche dans le cadre d'une instruction SQL.

Symbole	**Exemple**	**Utilisation**
*	*wh** trouve what, white et why **at* trouve cat, bat et what	Représente un nombre quelconque de caractères et peut être placé en tout point de la chaîne de caractères.
?	*b?ll* trouve ball, bell et bill	Représente un seul caractère.
#	*1#3* trouve 103, 113, 123	Représente un seul chiffre.
[ ]	*b[ae]ll* trouve ball et bell mais pas bill	Représente un seul caractère parmi ceux indiqués entre crochets.
!	*b[!ae]ll* trouve bill et bull mais pas bell ni ball	Représente tout caractère ne figurant pas entre crochets.
-	*b[a-c]d* trouve bad, bbd et bcd	Représente l'un des caractères de la plage indiquée.

Remarque
Les caractères * (astérisque), ? (point d'interrogation), # (dièse), [(crochet ouvrant) peuvent être considérés comme des caractères ordinaires dans une expression uniquement s'ils sont entourés de crochets.

- **Clause GROUP BY**

La clause GROUP BY combine en un seul enregistrement les enregistrements comportant des valeurs identiques dans la liste des champs spécifiés. Une valeur de synthèse est créée pour chaque enregistrement si vous incluez une ***fonction d'agrégation SQL***, par exemple **Sum** ou **Count**, dans l'instruction ***SELECT***.

Syntaxe

SELECT *listechamp*
FROM *table*
WHERE *critère*
[GROUP BY *grouplistechamp*]

Une instruction SELECT contenant une clause GROUP BY se compose des éléments suivants :

Élément	**Description**
listechamp	Noms du ou des champs à extraire avec leurs ***alias*** de nom de champ, ***fonctions d'agrégation SQL***, prédicats de sélection ***(ALL, DISTINCT ou TOP***) ou autres options d'instruction SELECT.
Table	Nom de la table contenant les enregistrements à extraire. Pour plus

	d'informations, reportez-vous à la rubrique relative à la clause ***FROM***.
Critère	Critères de sélection. Si l'instruction contient une clause ***WHERE***, SQL regroupe les valeurs après avoir appliqué les conditions WHERE aux enregistrements.
listechampgroupe	Noms des champs (10 au maximum) utilisés pour regrouper les enregistrements. L'ordre des noms de champs dans *listechampgroupe* détermine les niveaux de regroupement, du plus élevé au plus bas.

Notes

La clause GROUP BY est facultative.
Les valeurs de synthèse sont omises si l'instruction SELECT ne contient aucune fonction d'agrégation SQL.
Les valeurs ***Null*** des champs GROUP BY sont regroupées et ne sont pas omises. Toutefois, les valeurs **Null** ne sont évaluées dans aucune fonction d'agrégation SQL.

Utilisez la clause WHERE pour exclure les lignes que vous ne souhaitez pas regrouper

Tous les champs de la liste de champs SELECT doivent être inclus dans la clause GROUP BY ou, sous forme d'arguments, dans une fonction d'agrégation SQL.

Les fonctions d'agrégation

- **Fonctions First et Last**

Renvoient la valeur d'un champ du premier ou du dernier enregistrement contenu dans le jeu de résultats renvoyé par une requête.

Syntaxe
First(<*expression*>**)**
Last(<*expression*>**)**
L'argument <*expression*> représente une expression de chaîne qui identifie le champ contenant les données que vous souhaitez utiliser ou une expression qui exécute un calcul sur les données contenues dans ce champ. Les opérandes associés à <*expression*> peuvent comprendre le nom d'un champ de table, une constante ou une fonction (qui peut être soit intrinsèque, soit définie par l'utilisateur, à l'exclusion de toute autre fonction d'agrégation SQL).

- **Fonction Avg**

Calcule la moyenne arithmétique d'un ensemble de valeurs contenues dans un champ spécifié dans une requête.

Syntaxe
Avg(<*expression*>**)**
L'argument <*expression*> représente une expression de chaîne qui identifie le champ contenant les données numériques dont vous souhaitez calculer la moyenne ou une expression qui exécute un calcul sur la base des données contenues dans ce champ. Les opérandes associés à <*expression*> peuvent comprendre le nom d'un champ de table, une constante ou une fonction (qui peut être soit intrinsèque, soit définie par l'utilisateur, à l'exclusion de toute autre fonction d'agrégation SQL).

Remarques

La moyenne calculée par **Avg** est la moyenne arithmétique (la somme des valeurs divisée par le nombre de valeurs). Vous pouvez utiliser **Avg**, par exemple, pour calculer le coût moyen du transport des marchandises.
La fonction **Avg** n'intègre aucun champ **Null** à ses calculs.

- **Fonction Count**

Calcule le nombre d'enregistrements renvoyés par une requête.

Syntaxe
Count(<*expression*>**)**
L'argument <*expression*> représente une expression de chaîne qui identifie le champ contenant les données dont vous souhaitez obtenir le nombre ou une expression qui exécute un calcul sur les données contenues dans ce champ. Les opérandes associés à <*expression*> peuvent comprendre le nom d'un champ de table ou une fonction (qui peut être soit intrinsèque, soit définie par l'utilisateur, à l'exclusion de toute autre fonction d'agrégation SQL). Vous pouvez compter toutes sortes de données, y compris les données de type text.

Remarques
Vous pouvez utiliser la fonction **Count** pour compter le nombre d'enregistrements présents dans une requête sous-jacente. Par exemple, vous pouvez utiliser la fonction **Count** pour compter le nombre de commandes expédiées vers un pays donné.
Bien que l'argument <*expression*> puisse exécuter un calcul sur un champ, la fonction **Count** ne fait que dénombrer les enregistrements. Le type des valeurs contenues dans les enregistrements est sans importance.
La fonction **Count** ne compte pas les enregistrements possédant des champs **Null** sauf si l'argument <*expression*> est remplacé par le caractère générique astérisque (*). Si vous utilisez un astérisque, la fonction **Count** calcule le nombre total des enregistrements en incluant ceux qui contiennent des champs **Null**. **Count(*)** s'exécute beaucoup plus rapidement que **Count(**[*nom_de_colonne*]**)**. L'astérisque doit être entré sans apostrophes (' ').
Si <*expression*> identifie plusieurs champs, la fonction **Count** ne compte un enregistrement que si au moins un de ses champs n'est pas **Null**. Si tous les champs spécifiés sont **Null**, l'enregistrement n'est pas comptabilisé. Les noms des champs doivent être séparés par une perluète (&).

- **Fonction Sum**

Renvoie la somme d'une série de valeurs contenues dans un champ spécifié dans une requête.

Syntaxe
Sum(<*expression*>**)**
L'argument <*expression*> représente une expression de chaîne qui identifie le champ contenant les données numériques dont vous souhaitez calculer la somme ou une expression qui exécute un calcul sur la base des données contenues dans ce champ. Les opérandes associés à <*expression*> peuvent comprendre le nom d'un champ de table, une constante ou une fonction (qui peut être soit intrinsèque, soit définie par l'utilisateur, à l'exclusion de toute autre fonction d'agrégation SQL).

Remarques
La fonction **Sum** totalise les valeurs contenues dans un champ. Par exemple, vous pouvez utiliser la fonction **Sum** pour déterminer le coût total des frais de transport.
La fonction **Sum** ignore les enregistrements contenant des champs Null.

- **Fonctions Min et Max**

Renvoient la valeur minimale ou maximale d'une série de valeurs contenues dans un champ spécifié dans une requête.

Syntaxe
Min(<*expression*>**)**
Max(<*expression*>**)**
L'argument <*expression*> représente une expression de chaîne qui identifie le champ contenant les données que vous souhaitez évaluer ou une expression qui exécute un calcul sur les données contenues dans ce champ. Les opérandes associés à <*expression*> peuvent comprendre le nom d'un champ de table, une constante ou une fonction (qui peut être soit intrinsèque, soit définie par l'utilisateur, à l'exclusion de toute autre fonction d'agrégation SQL).

Remarques

Vous pouvez utiliser les fonctions **Min** et **Max** pour déterminer les valeurs les plus basses et les plus élevées contenues dans un champ en fonction du mode d'agrégation spécifié. Par exemple, vous pouvez utiliser ces fonctions pour renvoyer les coûts de transport des marchandises les plus bas et les plus élevés. Si aucune agrégation n'est spécifiée, toute la table est utilisée.

- **Clause HAVING**

Spécifie les enregistrements regroupés à afficher dans une instruction ***SELECT*** dotée d'une clause GROUP BY. Une fois que GROUP BY a regroupé les enregistrements, HAVING affiche tous les enregistrements qui ont été regroupés par la clause ***GROUP BY*** et qui répondent aux critères de la clause HAVING.

Syntaxe

SELECT *listechamp*
FROM *table*
WHERE *critèresélection*
GROUP BY *listechampgroupe*
[HAVING *critèregroupe]*

Une instruction SELECT contenant une clause HAVING se compose des éléments suivants :

Élément	**Description**
listechamp	Noms du ou des champs à extraire avec leurs ***alias*** de nom de champ, ***fonctions d'agrégation SQL***, prédicats de sélection (***ALL, DISTINCT ou TOP***) ou autres options d'instruction SELECT.
Table	Nom de la table contenant les enregistrements à extraire. Pour plus d'informations, reportez-vous à la rubrique relative à la clause ***FROM.***
critèresélection	Critères de sélection. Si l'instruction contient une clause ***WHERE***, le ***moteur de base de données Microsoft Jet*** regroupe les valeurs après avoir appliqué les conditions WHERE aux enregistrements.
listechampgroupe	Noms des champs (10 au maximum) utilisés pour regrouper les enregistrements. L'ordre des noms de champs dans *listechampgroupe* détermine les niveaux de regroupement, du plus élevé au plus bas.
critèregroupe	Expression qui détermine quels sont les enregistrements regroupés à afficher.

Notes

La clause HAVING est facultative.

HAVING est similaire à WHERE, qui détermine quels sont les enregistrements à sélectionner. Une fois que GROUP BY a regroupé les enregistrements, HAVING détermine quels sont les enregistrements qui seront affichés.
Une clause HAVING peut contenir jusqu'à 40 expressions liées par des opérateurs logiques comme **And** et **Or**.

- **Clause ORDER BY**

Trie, par ordre croissant ou décroissant, les enregistrements résultants d'une requête en fonction du ou des champs spécifiés.

Syntaxe

SELECT *listechamp*
FROM *table*
WHERE *critèressélection*
[ORDER BY *champ1* [ASC | DESC][, *champ2* [ASC | DESC]][, ...]]

Une instruction SELECT contenant une clause ORDER BY se compose des éléments suivants :

Élément	Description
listechamp	Noms du ou des champs à extraire avec leurs ***alias*** de nom de champ, ***fonctions d'agrégation SQL***, prédicats de sélection (***ALL, DISTINCT, ou TOP***) ou autres options d'instruction ***SELECT***.
Table	Nom de la table contenant les enregistrements à extraire. Pour plus d'informations, reportez-vous à la rubrique relative à la clause ***FROM***.
critèresélection	Critères de sélection. Si l'instruction contient une clause ***WHERE***, le ***moteur de base de données Microsoft Jet*** trie les valeurs après avoir appliqué les conditions WHERE aux enregistrements.
champ1, *champ2*	Noms des champs sur lesquels doit porter le tri.

Notes
La clause ORDER BY est facultative. Mais si vous n'incluez pas ORDER BY, les données sont affichées sans être triées.
L' ordre de tri par défaut est l'ordre croissant (de A à Z et de 0 à 9). Dans les deux exemples suivants, les employés sont triés sur leur nom :

```
SELECT Name, FirstName
FROM Employés
ORDER BY FirstName;
SELECT Name, FirstName
FROM Employés
ORDER BY FirstName ASC;
```

Pour effectuer un tri par ordre décroissant (de Z à A et de 9 à 0), ajoutez le mot réservé DESC à la fin de chaque champ que vous souhaitez trier par ordre décroissant. Dans les deux exemples suivants, les employés sont sélectionnés, puis triés par ordre décroissant :

SELECT Name, Salariés
FROM Employés
ORDER BY Salariés DESC, Name;

Si vous spécifiez un champ contenant des données de type ***Memo*** ou ***OLE Object*** dans la clause ORDER BY, une erreur se produit. Le moteur de base de données Microsoft Jet ne peut pas trier des champs contenant ces types de données.

ORDER BY est généralement le dernier élément d'une ***instruction SQL***.

Vous pouvez inclure des champs supplémentaires dans une clause ORDER BY. Les enregistrements sont triés tout d'abord en fonction du premier champ spécifié après ORDER BY. Ensuite, les enregistrements pour lesquels ce champ contient des valeurs identiques sont triés en fonction de la valeur du second champ spécifié, et ainsi de suite.

IV - SOUS-REQUETES SQL

Une sous-requête se compose d'une instruction ***SELECT*** imbriquée dans une autre instruction SELECT***, INSERT...INTO, DELETE*** ou ***UPDATE, etc.***, ou encore dans une autre sous-requête.

1. Les sous requêtes simples

Ce sont les sous requêtes utilisées dans la clause WHERE ou HAVING d'une requête pour fournir à celle-ci des informations de comparaison dont la requête principale a besoin pour s'exécuter

Syntaxe

Vous pouvez utiliser trois variantes de syntaxe pour créer une sous-requête simple :

WHERE ou HAVING <attribut> <opérateur de comparaison> [ANY | ALL | SOME] (*instructionsql*)
WHERE ou HAVING <attribut> [NOT] IN (*instructionsql*)
WHERE ou HAVING [NOT] EXISTS (*instructionsql*)

Une sous-requête comprend les éléments suivants :

Élément	Description
comparaison	***Expression*** et opérateur de comparaison qui compare l'expression avec les résultats de la sous-requête.
expression	Expression que l'on recherche dans le jeu de résultats de la sous-requête.
instructionsql	Instruction SELECT respectant le format et les règles conventionnelles des autres instructions SELECT. Elle doit être encadrée par des parenthèses.

Notes

Vous pouvez utiliser une sous-requête au lieu d'une expression, dans la liste de champs d'une instruction SELECT ou dans une clause ***WHERE*** ou ***HAVING***. Dans une sous-requête, vous utilisez une instruction SELECT pour fournir un jeu d'une ou plusieurs valeurs spécifiques à évaluer dans l'expression de la clause WHERE ou HAVING.

Utilisez les prédicats ANY ou SOME (l'effet est identique) pour rechercher par comparaison les enregistrements de la requête principale en correspondance avec n'importe quel enregistrement de la sous-requête. Dans l'exemple suivant, la requête renvoie tous les produits (products) dont le prix unitaire (UnitPrice) est supérieur au prix de n'importe quel produit vendu avec une remise (discount) de 25 pour cent ou davantage.

```
SELECT * FROM Products
WHERE UnitPrice > ANY
(SELECT UnitPrice FROM OrderDetails
WHERE Discount >= .25);
```

Utilisez le prédicat ***ALL*** pour rechercher par comparaison les enregistrements de la requête principale en correspondance avec tous les enregistrements de la sous-requête. Si dans l'exemple précédent, vous avez remplacé ANY par ALL, la requête ne renvoie que les produits dont le prix unitaire est supérieur aux prix de tous les produits vendus avec une remise de 25 pour cent ou davantage. La recherche est plus restrictive.

Utilisez le prédicat IN pour rechercher les enregistrements de la requête principale pour lesquels on trouve des enregistrements avec une valeur identique dans la sous-requête. Dans l'exemple suivant, la requête renvoie tous les produits vendus avec une remise de 25 pour cent ou davantage.

```
SELECT * FROM Products
WHERE ProductID IN
(SELECT ProductID FROM OrderDetails
WHERE Discount >= .25);
```

Inversement, vous pouvez utiliser NOT IN pour rechercher les enregistrements de la requête principale pour lesquels on ne trouve aucun enregistrement avec une valeur identique dans la sous-requête.

Utilisez le prédicat EXISTS (avec le mot réservé facultatif NOT) dans des comparaisons vrai/faux pour déterminer si la sous-requête renvoie des enregistrements.

Vous pouvez également utiliser des ***alias*** de nom de table dans une sous-requête, pour faire référence à des tables répertoriées dans une clause ***FROM*** située hors de la sous-requête. Dans l'exemple suivant, la requête renvoie les noms des employés dont les salaires sont supérieurs ou égaux au salaire moyen de l'ensemble des employés ayant la même qualité. L'alias affecté à la table Employés est "T1" :

```
SELECT Name
First name, Title, Salariés
FROM Employés AS T1
WHERE Salariés >=
(SELECT Avg(Salariés)
FROM Employés
WHERE T1.Title = Employés.Title) Order by Title;
```

Dans l'exemple précédent, le ***mot réservé*** AS est facultatif.

Certaines sous-requêtes sont autorisées dans les ***requêtes croisées*** mais uniquement comme prédicats (ceux de la clause WHERE). Les sous-requêtes ne sont pas autorisées dans les requêtes croisées pour ce qui concerne les sorties (dans la liste de SELECT).

2. Les Sous requêtes de la clause FROM

Elles sont utilisées pour fournir à une requête des enregistrements ne provenant pas d'une table.

Syntaxe

FROM <table1>, <Sous-requête> <alias>, …..

Soit les tables Etudiant et filière

Etudiant

Matricule	Nom	Sexe	Codfil
1	Magengo	M	IRT
2	Djangoni	F	MMI
3	Mandelai	F	MMI
4	Séchard	M	IRT
5	Mahigôh	M	MMI
6	Bitôh	F	IRT

Filière

CodFil	Libfil	Matricule(Responsable)
MMI	Management international	3
IRT	Ingénierie informatique et Réseau	6

Le matricule dans la table filière représente le matricule de l'étudiant responsable de la filière

On désire afficher pour chaque étudiant, son nom, le code de sa filière et le nom du responsable.

Le seul attribut Nom de la table Etudiant représente aussi bien les nom des étudiants que ceux des responsables. On est alors obligé de dissocier les responsables des autres étudiants à l'aide d'une requête avant de pouvoir les afficher distinctement.

Select E.Nom AS [Nom étudiant], R.Nom as [Nom respo], E.Codfil

From Etudiant E,(Select Nom, Codfil from Etudiant,filière Where Etudiant.Matricule=Filière.Matricule) R

Where E.Codfil=R.Codfil

Elle peut aussi être utilisée pour effectuer un comptage sans répétition (dans le cas d'une division par exemple)

3. L'Union, l'Intersection et la Différence

- **UNION**

L'objectif de la commande **UNION** de SQL est de combiner les résultats de deux requêtes. Une restriction de **UNION** est que toutes les colonnes correspondantes doivent inclure le même type de données. Aussi, lors de l'utilisation de **UNION**, seules les valeurs distinctes sont sélectionnées (similaire à **SELECT DISTINCT**).

La syntaxe est comme suit :

[instructions SQL 1]
UNION
[instructions SQL 2]

Supposons que nous avons les deux tables suivantes :

Table ***Etudiant***

Matricule	Nom	Prénom
1	Magengo	Guttembert
2	Djangoni	Mandelai
3	Plakaly	Monyo

Table ***Prof***

NumProf	Nom	Tel
1	Plakaly	65666768
2	Djangoni	95959897
3	Gansê	90959192
4	Madjrè	89988860

et pour retrouver les numéro et nom de tous les profs et les étudiants il faudra utiliser l'instruction SQL suivante :

SELECT Numprof, nom FROM Etudiant
UNION
SELECT Matricule, nom FROM Prof

Résultat :

Matricule	Nom
1	Magengo
2	Djangoni
3	Plakaly
1	Plakaly
3	Gansê
4	Madjrè

Notez qu'en saisissant « **SELECT DISTINCT**» pour les deux instructions SQL, nous obtiendrons le même jeu de résultats.

L'objectif de la commande **UNION ALL** de SQL est aussi de combiner ensemble les résultats de deux requêtes. La différence entre **UNION ALL** et **UNION** est la suivante : **UNION** sélectionne seulement des valeurs distinctes et **UNION ALL** sélectionne toutes les valeurs.

- **INTERSECT (Intersection en algèbre relationel)**

À l'instar de la commande **UNION**, **INTERSECT** opère aussi sur deux instructions SQL. La différence entre les deux commandes est la suivante : **UNION** agit essentiellement comme un opérateur **OR (OU)** operator (la valeur est sélectionnée si elle apparaît dans la première ou la deuxième instruction), la

commande **INTERSECT** agit comme un opérateur **AND (ET)** operator (la valeur est sélectionnée seulement si elle apparaît dans les deux instructions).

La syntaxe est comme suit :

[instructions SQL 1]
INTERSECT
[instructions SQL 2]

En utilisant les deux tables précédentes et pour retrouver tous les étudiants qui sont encore des profs, il faudra utiliser l'instruction SQL suivante :

SELECT Numprof, nom FROM Etudiant
INTERSECT
SELECT Matricule, nom FROM Prof

Résultat :

NumProf	Nom
2	Djangoni

Donner le résultat de la requête :

SELECT nom FROM Etudiant
INTERSECT
SELECT nom FROM Prof

Notez que la commande **INTERSECT** ne retournera que des valeurs distinctes.

- **MINUS (différence en algèbre relationnelle)**

MINUS opère sur deux instructions SQL. Elle prend tous les résultats de la première instruction SQL, puis soustrait ceux de la deuxième instruction SQL pour obtenir la réponse finale. Si la deuxième instruction SQL comprend des résultats qui ne sont pas inclus dans la première instruction SQL, ils seront ignorés.

La syntaxe est comme suit :

[instructions SQL 1]
MINUS
[instructions SQL 2]

Poursuivons avec le même exemple pour trouver tous les étudiants qui ne sont pas des profs, il faudra utiliser l'instruction SQL suivante :

SELECT Numprof, nom FROM Etudiant
MINUS
SELECT Matricule, nom FROM Prof

Notez que la commande **MINUS** ne retournera que des valeurs distinctes.